

# AMERICAN INTERNATIONAL UNIVERSITY-BANGLADESH

408/1, Kuratoli, Khilkhet, Dhaka 1229, Bangladesh



Project Name: Make-up class management system

| No | Name                  | ID         | Program | Contribution                                           |
|----|-----------------------|------------|---------|--------------------------------------------------------|
| 1  | MD REDWANUL HOQUE     | 23-50057-1 | CSE     | 25% Updates, Schema diagram, Relation Algebra, Website |
| 2  | DIPU ROY              | 23-50420-1 | CSE     | 25% Exception Handling, Normalization                  |
| 3  | DIPTA MONI SINHA      | 23-54437-3 | CSE     | 25% Exception Handling                                 |
| 4  | AKHTAB, SHAH MD. AKIL | 21-44736-1 | CSE     | 25% Implicit Locking, Explicit Locking                 |

Course Name: Advance database management system

Section: B

## Project Updates:

## Normalization:

### 0. Entities and cardinality assumptions

| Relationship                                        | Reading                                     | Cardinality                       |
|-----------------------------------------------------|---------------------------------------------|-----------------------------------|
| Teacher <b>have</b> Department                      | a department has many teachers              | Teacher (N) :<br>Department (1)   |
| Department <b>headed by</b> HOD                     | one department, one head                    | Department (1) : HOD<br>(1)       |
| Teacher <b>teach</b> Class                          | a teacher teaches many classes              | Class (N) : Teacher (1)           |
| Class <b>have</b> Student                           | students enroll in many classes             | Class (M) : Student (N)           |
| Class <b>request</b> Make-up class request          | a class can be rescheduled more than once   | Class (1) : Request (N)           |
| Make-up class request <b>approved by</b> HOD        | one HOD approves many requests              | Request (N) : HOD (1)             |
| Make-up class request <b>held in</b> Classroom      | a room hosts many requests over time        | Request (N) : Classroom<br>(1)    |
| Make-up class request <b>generates</b> Notification | a request can trigger more than one notice  | Request (1) :<br>Notification (N) |
| Notification <b>sent to</b> Student                 | a notice goes to every student in the class | Notification (M) :<br>Student (N) |

Entity attributes (from the diagram), with primary keys underlined in the source:

- **Teacher:** id (PK), name, designation, email, address, phone\_number (double-circle → multi-valued)
- **Department:** id (PK), name, phone
- **HOD:** id (PK), name, phone, email
- **Class:** id (PK), course\_name, section, schedule
- **Make-up class request:** id (PK), request\_date, proposed\_time, approved\_date, status
- **Student:** id (PK), name, address, email, phone\_number (double-circle → multi-valued)
- **Classroom:** id (PK), building\_name, capacity
- **Notification:** notification\_id (PK), message, date\_time

### 1. Teacher have Department ( $N : 1$ )

**UNF:** have(t\_id, t\_name, designation, email, address, phone\_number, d\_id, d\_name, d\_phone)

**1NF** — phone\_number is multi-valued, pull it out:

1. t\_id, t\_name, designation, email, address, d\_id, d\_name, d\_phone
2. t\_id, phone\_number

**2NF** — key of relation 1 is t\_id alone (single attribute, since many teachers share one department), so there's no *partial* dependency to remove. Unchanged.

**3NF** —  $t\_id \rightarrow d\_id \rightarrow d\_name, d\_phone$  is a transitive dependency (department attributes depend on d\_id, not directly on t\_id). Split it out:

1. t\_id, t\_name, designation, email, address, d\_id (FK)
2. d\_id, d\_name, d\_phone
3. t\_id, phone\_number

### 2. Department headed by HOD ( $1 : 1$ )

**UNF:** head(d\_id, d\_name, d\_phone, h\_id, h\_name, h\_phone, h\_email)

**1NF** — no multi-valued attributes. Unchanged.

**2NF** — this is exactly the head example from your slides: split department attributes from HOD attributes (department name/phone belong to d\_id; HOD name/phone/email belong to h\_id):

1. d\_id, d\_name, d\_phone
2. h\_id, h\_name, h\_phone, h\_email

**3NF** — no transitive dependency left. Unchanged.

### 3. Teacher teach Class ( $N : 1$ )

**UNF:** teach(c\_id, course\_name, section, schedule, t\_id, t\_name, designation, email, address, phone\_number)

**1NF** — remove multi-valued phone\_number:

1. c\_id, course\_name, section, schedule, t\_id, t\_name, designation, email, address
2. t\_id, phone\_number

**2NF** — key of relation 1 is `c_id` alone (many classes, one teacher each) → no partial dependency. Unchanged.

**3NF** — `c_id` → `t_id` → `t_name`, `designation`, `email`, `address` is transitive. Split:

1. `c_id`, `course_name`, `section`, `schedule`, `t_id` (FK)
2. `t_id`, `t_name`, `designation`, `email`, `address`
3. `t_id`, `phone_number`

#### 4. Class have Student (*M : N*)

**UNF**: `enroll(c_id, course_name, section, schedule, s_id, s_name, address, email, phone_number)`

**1NF** — remove multi-valued `phone_number`:

1. `c_id`, `course_name`, `section`, `schedule`, `s_id`, `s_name`, `address`, `email`
2. `s_id`, `phone_number`

**2NF** — because the relationship is M:N, the real key of relation 1 is the **composite** (`c_id`, `s_id`). `course_name`, `section`, `schedule` depend only on `c_id` (partial dependency); `s_name`, `address`, `email` depend only on `s_id` (partial dependency). Remove both:

1. `c_id`, `course_name`, `section`, `schedule`
2. `s_id`, `s_name`, `address`, `email`
3. `c_id`, `s_id` — the enrollment bridge table itself
4. `s_id`, `phone_number`

**3NF** — each relation is already a single atomic entity; nothing transitive left. Unchanged.

#### 5. Class request Make-up class request (*1 : N*)

**UNF**: `request(r_id, request_date, proposed_time, approved_date, status, c_id, course_name, section, schedule)`

**1NF** — no multi-valued attributes. Unchanged.

**2NF** — key is `r_id` alone (many requests per class) → no partial dependency. Unchanged.

**3NF** — `r_id` → `c_id` → `course_name`, `section`, `schedule` is transitive. Split:

1. `r_id`, `request_date`, `proposed_time`, `approved_date`, `status`, `c_id` (FK)
2. `c_id`, `course_name`, `section`, `schedule`

#### 6. Make-up class request approved by HOD (*N : 1*)

**UNF:** approve(r\_id, request\_date, proposed\_time, approved\_date, status, h\_id, h\_name, h\_phone, h\_email)

**1NF** — unchanged (atomic).

**2NF** — key r\_id alone → no partial dependency.

**3NF** — r\_id → h\_id → h\_name, h\_phone, h\_email is transitive. Split:

1. r\_id, request\_date, proposed\_time, approved\_date, status, h\_id (FK)
2. h\_id, h\_name, h\_phone, h\_email

### **7. Make-up class request held in Classroom (*N : 1*)**

**UNF:** held\_in(r\_id, request\_date, proposed\_time, approved\_date, status, room\_id, building\_name, capacity)

**1NF** — unchanged.

**2NF** — key r\_id alone → no partial dependency.

**3NF** — r\_id → room\_id → building\_name, capacity is transitive. Split:

1. r\_id, request\_date, proposed\_time, approved\_date, status, room\_id (FK)
2. room\_id, building\_name, capacity

### **8. Make-up class request generates Notification (*1 : N*)**

**UNF:** generates(n\_id, message, date\_time, r\_id, request\_date, proposed\_time, approved\_date, status)

**1NF** — unchanged.

**2NF** — key n\_id alone (many notifications per request) → no partial dependency.

**3NF** — n\_id → r\_id → request\_date, proposed\_time, approved\_date, status is transitive. Split:

1. n\_id, message, date\_time, r\_id (FK)
2. r\_id, request\_date, proposed\_time, approved\_date, status

### **9. Notification sent to Student (*M : N*)**

**UNF:** sent\_to(n\_id, message, date\_time, s\_id, s\_name, address, email, phone\_number)

**1NF** — remove multi-valued phone\_number:

1. n\_id, message, date\_time, s\_id, s\_name, address, email
2. s\_id, phone\_number

**2NF** — M:N → composite key (n\_id, s\_id). message, date\_time depend only on n\_id; s\_name, address, email depend only on s\_id. Split:

1. n\_id, message, date\_time
2. s\_id, s\_name, address, email
3. n\_id, s\_id — the notification-recipient bridge table
4. s\_id, phone\_number

**3NF** — already atomic. Unchanged.

Temporary Tables:

1. t\_id, t\_name, designation, email, address, d\_id (FK)
2. d\_id, d\_name, d\_phone
3. t\_id, phone\_number
4. ~~d\_id, d\_name, d\_phone~~
5. h\_id, h\_name, h\_phone, h\_email
6. c\_id, course\_name, section, schedule, t\_id (FK)
7. ~~t\_id, t\_name, designation, email, address~~
8. ~~t\_id, phone\_number~~
9. c\_id, course\_name, section, schedule
10. s\_id, s\_name, address, email
11. c\_id, s\_id — the enrollment bridge table itself
12. s\_id, phone\_number
13. r\_id, request\_date, proposed\_time, approved\_date, status, c\_id (FK)
14. ~~c\_id, course\_name, section, schedule~~
15. r\_id, request\_date, proposed\_time, approved\_date, status, h\_id (FK)
16. ~~h\_id, h\_name, h\_phone, h\_email~~
17. r\_id, request\_date, proposed\_time, approved\_date, status, room\_id (FK)
18. room\_id, building\_name, capacity
19. n\_id, message, date\_time, r\_id (FK)
20. ~~r\_id, request\_date, proposed\_time, approved\_date, status~~
21. ~~n\_id, message, date\_time~~
22. ~~s\_id, s\_name, address, email~~
23. n\_id, s\_id — the notification-recipient bridge table
24. ~~s\_id, phone\_number~~

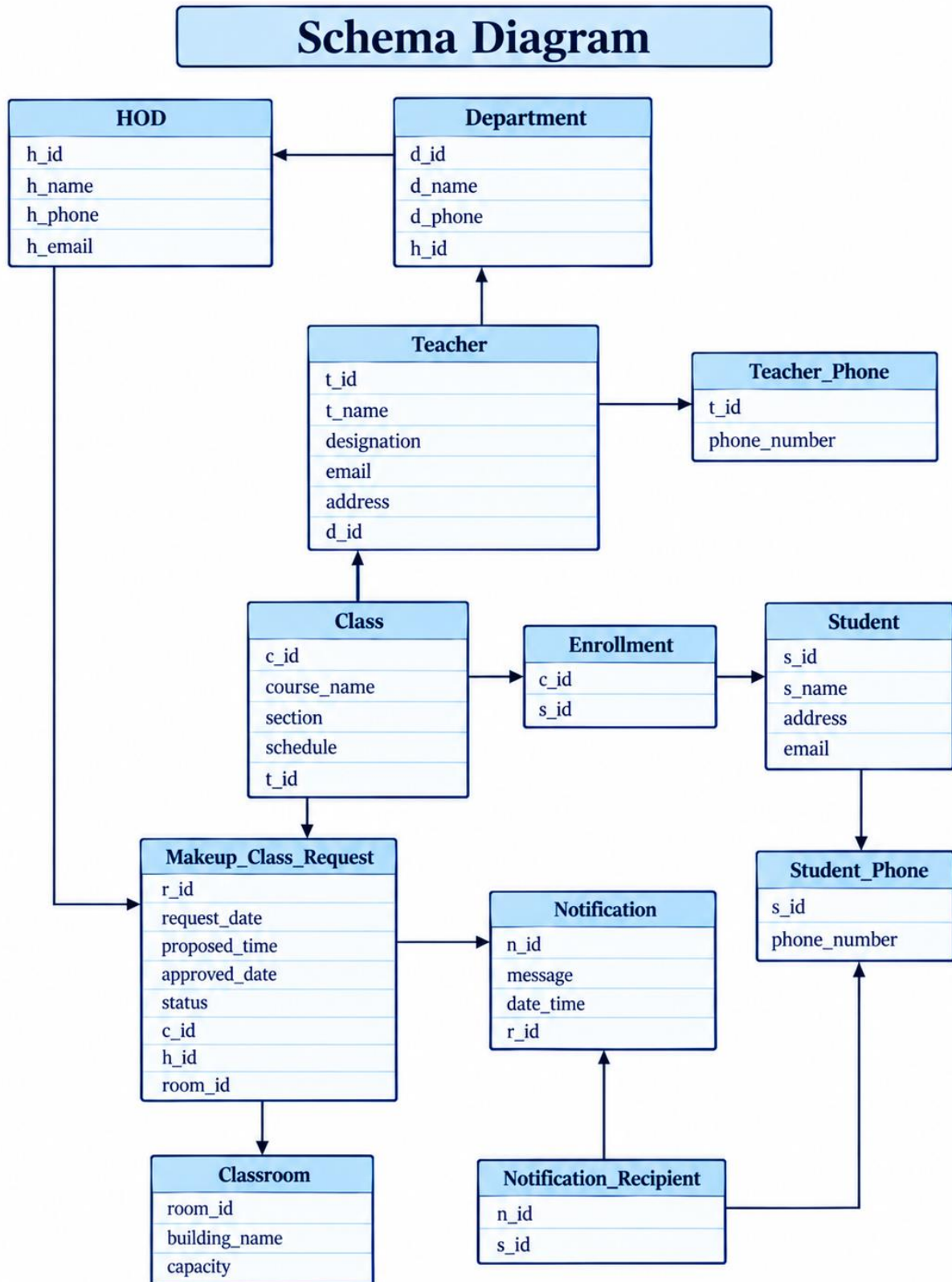
## Final Tables:

1. t\_id, t\_name, designation, email, address, d\_id (FK)
2. d\_id, d\_name, d\_phone
3. t\_id, phone\_number
4. h\_id, h\_name, h\_phone, h\_email
5. c\_id, course\_name, section, schedule, t\_id (FK)
6. c\_id, course\_name, section, schedule
7. s\_id, s\_name, address, email
8. c\_id, s\_id — Enrollment bridge
9. s\_id, phone\_number
10. r\_id, request\_date, proposed\_time, approved\_date, status, c\_id (FK)
11. r\_id, request\_date, proposed\_time, approved\_date, status, h\_id (FK)
12. r\_id, request\_date, proposed\_time, approved\_date, status, room\_id (FK)
13. room\_id, building\_name, capacity
14. n\_id, message, date\_time, r\_id (FK)
15. n\_id, s\_id — Notification-Recipient bridge

## 10. Merge the repeated tables

| No. Final Table                  | Attributes                                                                                   |
|----------------------------------|----------------------------------------------------------------------------------------------|
| 1 <b>Department</b>              | d_id PK, d_name, d_phone, h_id FK                                                            |
| 2 <b>HOD</b>                     | h_id PK, h_name, h_phone, h_email                                                            |
| 3 <b>Teacher</b>                 | t_id PK, t_name, designation, email, address, d_id FK                                        |
| 4 <b>Teacher_Phone</b>           | t_id FK, phone_number — PK(t_id, phone_number)                                               |
| 5 <b>Class</b>                   | c_id PK, course_name, section, schedule, t_id FK                                             |
| 6 <b>Student</b>                 | s_id PK, s_name, address, email                                                              |
| 7 <b>Student_Phone</b>           | s_id FK, phone_number — PK(s_id, phone_number)                                               |
| 8 <b>Enrollment</b>              | c_id FK, s_id FK — PK(c_id, s_id)                                                            |
| 9 <b>Makeup_Class_Request</b>    | r_id PK, request_date, proposed_time, approved_date, status,<br>c_id FK, h_id FK, room_id FK |
| 10 <b>Classroom</b>              | room_id PK, building_name, capacity                                                          |
| 11 <b>Notification</b>           | n_id PK, message, date_time, r_id FK                                                         |
| 12 <b>Notification_Recipient</b> | n_id FK, s_id FK — PK(n_id, s_id)                                                            |

## Schema Diagram:

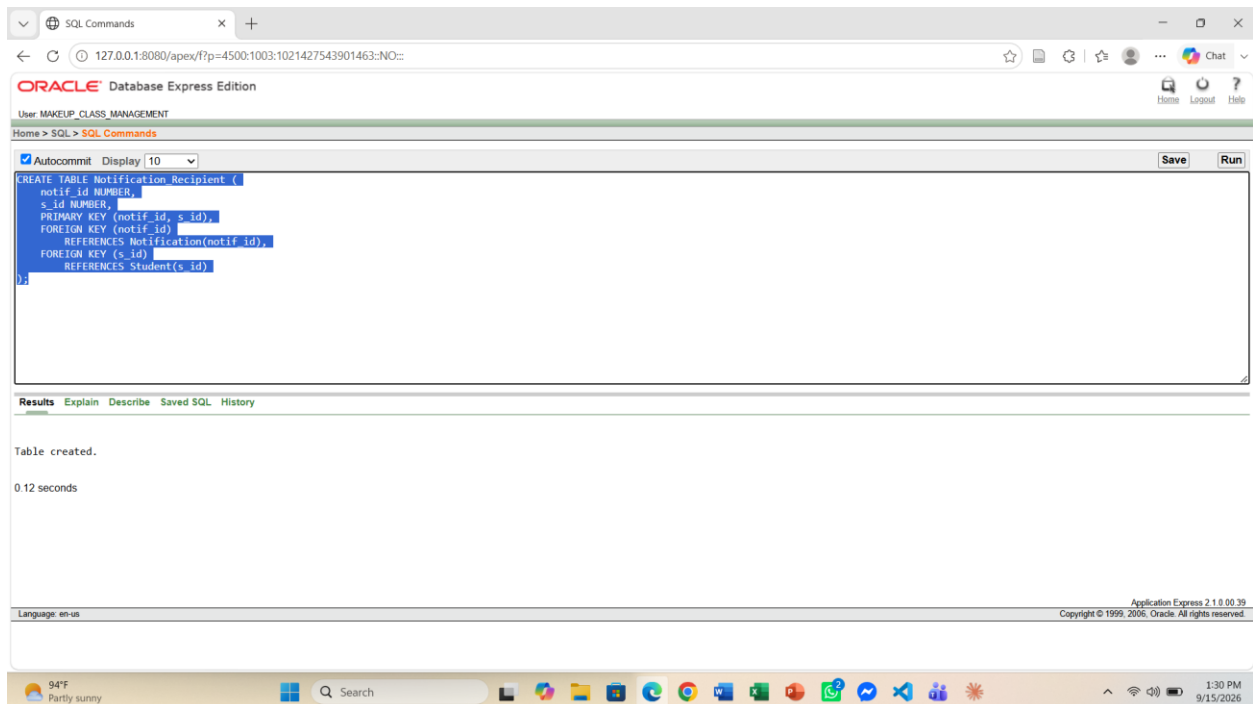


## New tables find out the modify normalization (Notification\_Recipient)

### Table creation using SQL:

- **Notification\_Recipient**

```
CREATE TABLE Notification_Recipient (  
    notif_id NUMBER,  
    s_id NUMBER,  
    PRIMARY KEY (notif_id, s_id),  
    FOREIGN KEY (notif_id)  
        REFERENCES Notification(notif_id),  
    FOREIGN KEY (s_id)  
        REFERENCES Student(s_id)  
);
```



## Data Insertion Using SQL:

- **Notification\_Recipient**

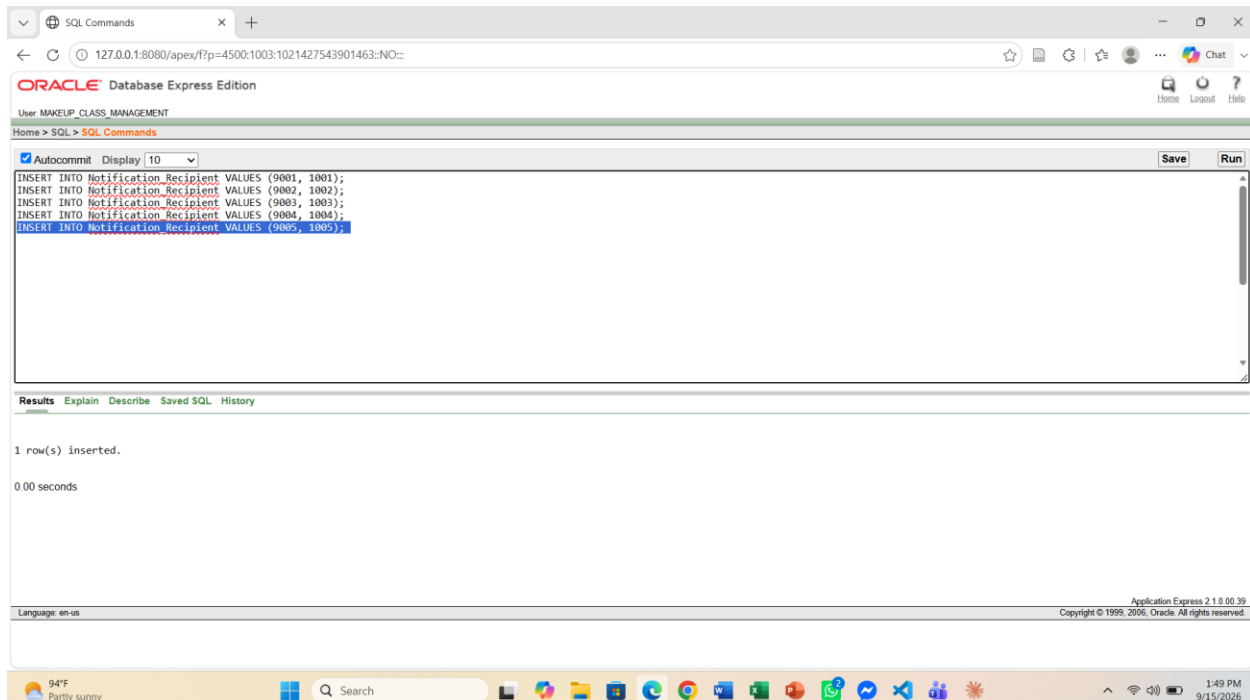
INSERT INTO Notification\_Recipient VALUES (9001, 1001);

INSERT INTO Notification\_Recipient VALUES (9002, 1002);

INSERT INTO Notification\_Recipient VALUES (9003, 1003);

INSERT INTO Notification\_Recipient VALUES (9004, 1004);

INSERT INTO Notification\_Recipient VALUES (9005, 1005);



## Table data show:

### 1. HOD Table:

Select \* from HOD;

The screenshot shows the Oracle Database Express Edition interface. The SQL Commands window contains the query `select * from HOD;`. The Results window displays the following data:

| HOD_ID | HOD_NAME   | HOD_EMAIL       | HOD_PHONE   |
|--------|------------|-----------------|-------------|
| 1      | Dr. Rahman | rahman@alub.edu | 01710000001 |
| 2      | Dr. Karim  | karim@alub.edu  | 01710000002 |
| 3      | Dr. Alam   | alam@alub.edu   | 01710000003 |
| 4      | Dr. Hasan  | hasan@alub.edu  | 01710000004 |
| 5      | Dr. Nayeem | nayeem@alub.edu | 01710000005 |
| 6      | Dr. Redwan | redwan@alub.edu | 01810000006 |
| 7      | Dr. Rishad | rishad@alub.edu | 01810000007 |

7 rows returned in 0.02 seconds

### 2. Department Table:

Select \* from Department;

The screenshot shows the Oracle Database Express Edition interface. The SQL Commands window contains the query `select * from Department;`. The Results window displays the following data:

| DEPT_ID | DEPT_NAME | DEPT_PHONE | HOD_ID |
|---------|-----------|------------|--------|
| 101     | CSE       | 0288000001 | 1      |
| 102     | EEE       | 0288000002 | 2      |
| 103     | BBA       | 0288000003 | 3      |
| 104     | CS        | 0288000004 | 4      |
| 105     | SE        | 0288000005 | 5      |

5 rows returned in 0.02 seconds

### 3. Teacher Table:

Select \* from Teacher;

The screenshot shows the Oracle Database Express Edition interface. The SQL command 'Select \* from Teacher;' is entered in the SQL Commands window. The results are displayed in a table with 5 rows and 6 columns: T\_ID, T\_NAME, DESIGNATION, T\_ADDRESS, T\_EMAIL, and DEPT\_ID.

| T_ID | T_NAME | DESIGNATION         | T_ADDRESS | T_EMAIL         | DEPT_ID |
|------|--------|---------------------|-----------|-----------------|---------|
| 1    | Anis   | Professor           | Dhaka     | anis@alub.edu   | 101     |
| 2    | Nadia  | Associate Professor | Dhaka     | nadia@alub.edu  | 102     |
| 3    | Sabbir | Lecturer            | Dhaka     | sabbir@alub.edu | 103     |
| 4    | Rima   | Assistant Professor | Dhaka     | rima@alub.edu   | 104     |
| 5    | Tariq  | Lecturer            | Dhaka     | tariq@alub.edu  | 105     |

5 rows returned in 0.00 seconds

### 4. TeacherPhone Table:

Select \* from TeacherPhone;

The screenshot shows the Oracle Database Express Edition interface. The SQL command 'Select \* from TeacherPhone;' is entered in the SQL Commands window. The results are displayed in a table with 5 rows and 2 columns: T\_ID and T\_PHONE.

| T_ID | T_PHONE     |
|------|-------------|
| 1    | 01711111111 |
| 2    | 01722222222 |
| 3    | 01733333333 |
| 4    | 01744444444 |
| 5    | 01755555555 |

5 rows returned in 0.02 seconds

## 5. Class Table:

Select \* from Class;

The screenshot shows the Oracle Database Express Edition interface. The SQL Commands window contains the query `select * from class;`. The Results window displays the following data:

| CLASS_ID | COURSE_NAME | SECTION | SCHEDULE | T_ID |
|----------|-------------|---------|----------|------|
| 201      | DBMS        | B       | Sun 10AM | 1    |
| 202      | OOP         | A       | Mon 11AM | 2    |
| 203      | Networks    | C       | Tue 9AM  | 3    |
| 204      | AI          | A       | Wed 2PM  | 4    |
| 205      | Compiler    | B       | Thu 1PM  | 5    |

5 rows returned in 0.02 seconds [CSV Export](#)

Language: en-us Application Express 2.1.8.00.39 Copyright © 1999, 2006, Oracle. All rights reserved.

## 6. Student table:

Select \* from Student;

SQL Commands

127.0.0.1:8080/apex/f?p=4500:1003:1021427543901463::NO::

ORACLE Database Express Edition

User MAKEUP\_CLASS\_MANAGEMENT

Home > SQL > SQL Commands

Autocommit Display 10 Save Run

```
select * from Student;
```

Results Explain Describe Saved SQL History

| S_ID | S_NAME | S_ADDRESS | CLASS_ID |
|------|--------|-----------|----------|
| 1001 | Rakib  | Dhaka     | 201      |
| 1002 | Rifat  | Dhaka     | 202      |
| 1003 | Sadia  | Gazipur   | 203      |
| 1004 | Nabila | Narsingdi | 204      |
| 1005 | Sohan  | Dhaka     | 205      |

5 rows returned in 0.02 seconds [CSV Export](#)

Language: en-us

Application Express 2.1.8.00.39  
Copyright © 1999, 2006, Oracle. All rights reserved.

94°F Partly sunny 2:10 PM 9/15/2026

## 7. StudentEmail table:

Select \* from StudentEmail;

SQL Commands

127.0.0.1:8080/apex/f?p=4500:1003:1021427543901463::NO::

ORACLE Database Express Edition

User MAKEUP\_CLASS\_MANAGEMENT

Home > SQL > SQL Commands

Autocommit Display 10 Save Run

```
select * from StudentEmail;
```

Results Explain Describe Saved SQL History

| S_ID | S_EMAIL          |
|------|------------------|
| 1001 | rakib@gmail.com  |
| 1002 | rifat@gmail.com  |
| 1003 | sadia@gmail.com  |
| 1004 | nabila@gmail.com |
| 1005 | redwma@gmail.com |

5 rows returned in 0.00 seconds [CSV Export](#)

Language: en-us

Application Express 2.1.8.00.39  
Copyright © 1999, 2006, Oracle. All rights reserved.

94°F Partly sunny 2:12 PM 9/15/2026

## 8. StudentPhone table:

Select \* from StudentPhone;

The screenshot shows the Oracle Database Express Edition interface. The SQL Commands window contains the query `select * from StudentPhone;`. The Results window displays the following data:

| S_ID | S_PHONE     |
|------|-------------|
| 1001 | 01811111111 |
| 1002 | 01822222222 |
| 1003 | 01833333333 |
| 1004 | 01844444444 |
| 1005 | 15874689364 |

5 rows returned in 0.00 seconds. The interface also shows the user `MAKELP_CLASS_MANAGEMENT` and the application version `2.1.0.00.39`.

## 9. Classroom table:

select \* from Classroom ;

The screenshot shows the Oracle Database Express Edition interface. The SQL Commands window contains the query `select * from Classroom;`. The Results window displays the following data:

| ROOM_ID | BUILDING_NAME | CAPACITY |
|---------|---------------|----------|
| 1       | Building A    | 40       |
| 2       | Building B    | 50       |
| 3       | Building C    | 35       |
| 4       | Building D    | 60       |
| 5       | Building E    | 45       |

5 rows returned in 0.02 seconds. The interface also shows the user `MAKELP_CLASS_MANAGEMENT` and the application version `2.1.0.00.39`.

## 10. MakeupClassRequest table:

select \* from MakeupClassRequest;

The screenshot shows the Oracle Database Express Edition interface. The SQL command 'select \* from MakeupClassRequest;' has been executed. The results are displayed in a table with 5 rows and 8 columns: REQ\_ID, REQUESTED\_DATE, PROPOSED\_TIME, STATUS, APPROVED\_DATE, CLASS\_ID, ROOM\_ID, and MOD\_ID. The status of the requests varies, including 'Approved', 'Pending', and 'Rejected'.

| REQ_ID | REQUESTED_DATE | PROPOSED_TIME | STATUS   | APPROVED_DATE | CLASS_ID | ROOM_ID | MOD_ID |
|--------|----------------|---------------|----------|---------------|----------|---------|--------|
| 501    | 01-AUG-26      | 10:00 AM      | Approved | 30-JUL-26     | 201      | 1       | 1      |
| 502    | 02-AUG-26      | 11:00 AM      | Pending  | -             | 202      | 2       | 2      |
| 503    | 03-AUG-26      | 09:00 AM      | Approved | 01-AUG-26     | 203      | 3       | 3      |
| 504    | 04-AUG-26      | 02:00 PM      | Rejected | -             | 204      | 4       | 4      |
| 505    | 05-AUG-26      | 01:00 PM      | Approved | 03-AUG-26     | 205      | 5       | 5      |

5 rows returned in 0.01 seconds

## 11. Notification table:

select \* from Notification ;

The screenshot shows the Oracle Database Express Edition interface. The SQL command 'select \* from Notification ;' has been executed. The results are displayed in a table with 5 rows and 5 columns: NOTIF\_ID, MESSAGE, SENT\_DATETIME, REQ\_ID, and S\_ID. The messages provide details about the approval status of the makeup class requests.

| NOTIF_ID | MESSAGE                        | SENT_DATETIME                | REQ_ID | S_ID |
|----------|--------------------------------|------------------------------|--------|------|
| 9001     | Your make-up class is approved | 30-JUL-26 10:00:00.000000 AM | 501    | 1001 |
| 9002     | Await approval                 | 30-JUL-26 11:00:00.000000 AM | 502    | 1002 |
| 9003     | Approved                       | 01-AUG-26 09:00:00.000000 AM | 503    | 1003 |
| 9004     | Rejected                       | 02-AUG-26 10:00:00.000000 AM | 504    | 1004 |
| 9005     | Approved                       | 03-AUG-26 11:00:00.000000 AM | 505    | 1005 |

5 rows returned in 0.02 seconds

## 12. Notification\_Recipient table:

select \* from Notification\_Recipient;

The screenshot shows the Oracle Database Express Edition interface. The SQL Commands window contains the query: `select * from Notification_Recipient;`. The query has been executed, and the results are displayed in a table with two columns: `NOTIF_ID` and `S_ID`. The results show five rows of data. Below the table, it indicates "5 rows returned in 0.00 seconds" and provides a "CSV Export" link. The interface also includes tabs for "Results", "Explain", "Describe", "Saved SQL", and "History". The bottom of the window shows the system tray with weather information (93°F, Mostly sunny) and the date/time (2:25 PM, 9/15/2026).

| NOTIF_ID | S_ID |
|----------|------|
| 9001     | 1001 |
| 9002     | 1002 |
| 9003     | 1003 |
| 9004     | 1004 |
| 9005     | 1005 |

Implicit & Explicit locking:

Query: Write a commit in the classroom table

The image displays two screenshots of a Windows Command Prompt running SQL\*Plus, illustrating database operations and error handling.

**Top Screenshot:**

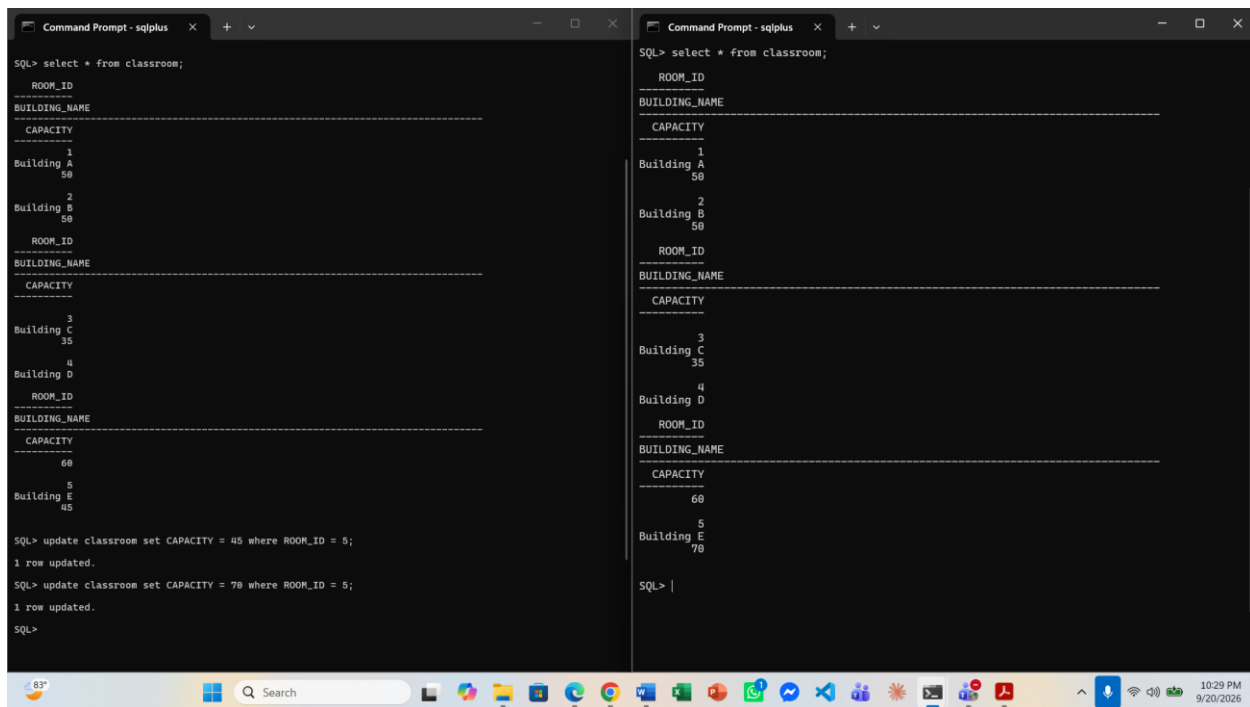
- Left Window:** Shows an attempt to update the capacity of a classroom using a variable `v_capacity`. The first attempt fails with the error `ORA-00904: "V_CAPACITY": invalid identifier`. The second attempt, using `capacity` instead of `v_capacity`, succeeds, updating 1 row.
- Right Window:** Shows the output of the `select * from classroom;` query, displaying the current state of the `classroom` table.

| ROOM_ID | BUILDING_NAME | CAPACITY |
|---------|---------------|----------|
| 1       | Building A    | 50       |
| 2       | Building B    | 50       |
| 3       | Building C    | 35       |
| 4       | Building D    | 60       |
| 5       | Building E    | 70       |

**Bottom Screenshot:**

- Left Window:** Shows the same update operation as the top screenshot, but followed by a `COMMIT;` statement, which completes successfully.
- Right Window:** Shows the output of the `select * from classroom;` query after the commit, showing the updated capacity for Building A (50) and Building B (50).

| ROOM_ID | BUILDING_NAME | CAPACITY |
|---------|---------------|----------|
| 1       | Building A    | 50       |
| 2       | Building B    | 50       |
| 3       | Building C    | 35       |
| 4       | Building D    | 60       |
| 5       | Building E    | 70       |



The image shows two side-by-side SQL command prompt windows. The left window displays the output of a 'select \* from classroom;' query, showing five buildings with their room IDs and capacities. It then shows two 'update' statements for room 5, changing its capacity from 45 to 70. The right window shows the same 'select \* from classroom;' query, but the output for room 5 now shows a capacity of 70.

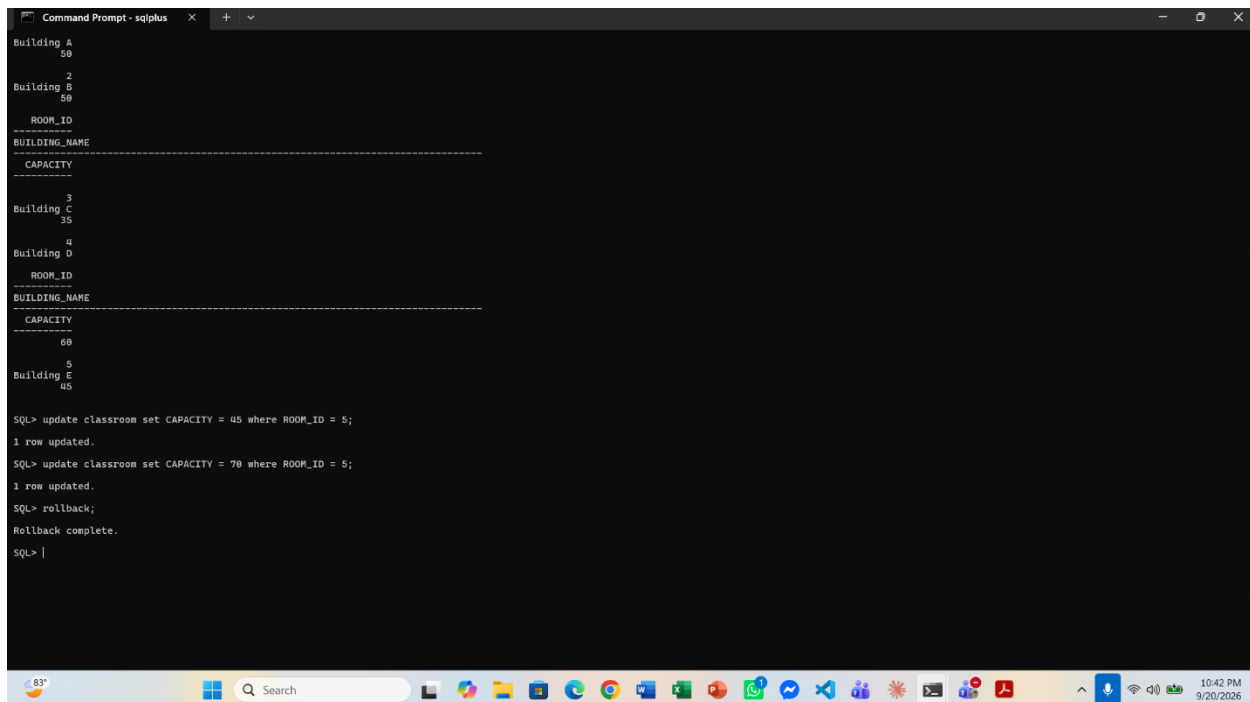
```
SQL> select * from classroom;
  ROOM_ID
-----
BUILDING_NAME
-----
CAPACITY
-----
1
Building A
50
2
Building B
50
  ROOM_ID
-----
BUILDING_NAME
-----
CAPACITY
-----
3
Building C
35
4
Building D
  ROOM_ID
-----
BUILDING_NAME
-----
CAPACITY
-----
60
5
Building E
45

SQL> update classroom set CAPACITY = 45 where ROOM_ID = 5;
1 row updated.
SQL> update classroom set CAPACITY = 70 where ROOM_ID = 5;
1 row updated.
SQL>
```

```
SQL> select * from classroom;
  ROOM_ID
-----
BUILDING_NAME
-----
CAPACITY
-----
1
Building A
50
2
Building B
50
  ROOM_ID
-----
BUILDING_NAME
-----
CAPACITY
-----
3
Building C
35
4
Building D
  ROOM_ID
-----
BUILDING_NAME
-----
CAPACITY
-----
60
5
Building E
70

SQL> |
```

Query: Write a rollback in the classroom table



The image shows a single SQL command prompt window. It displays the same 'select \* from classroom;' query output as before. Then, it shows an 'update' statement for room 5, followed by a 'rollback;' statement. The output confirms that the update was successful and then that the rollback was complete.

```
Building A
50
2
Building B
50
  ROOM_ID
-----
BUILDING_NAME
-----
CAPACITY
-----
3
Building C
35
4
Building D
  ROOM_ID
-----
BUILDING_NAME
-----
CAPACITY
-----
60
5
Building E
45

SQL> update classroom set CAPACITY = 45 where ROOM_ID = 5;
1 row updated.
SQL> update classroom set CAPACITY = 70 where ROOM_ID = 5;
1 row updated.
SQL> rollback;
Rollback complete.
SQL> |
```

## Implicit Locking:

### Query 1: Update Make-up Class Request Status

#### Question:

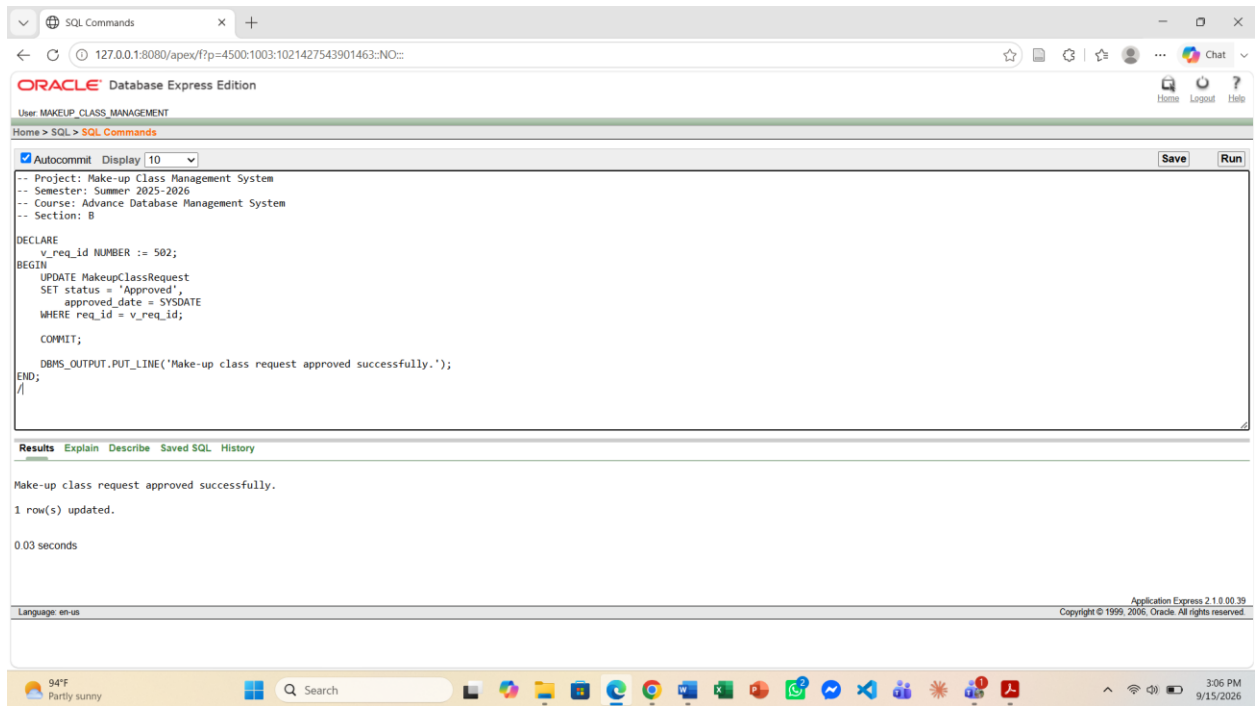
Write a PL/SQL program using implicit locking to update the status of a Make-up Class Request.

```
DECLARE
    v_req_id NUMBER := 502;
BEGIN
    UPDATE MakeupClassRequest
    SET status = 'Approved',
        approved_date = SYSDATE
    WHERE req_id = v_req_id;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Make-up class request approved successfully.');
```

END;  
/



## Query 2: Update Classroom Capacity

### Question:

Write a PL/SQL program using implicit locking to update the capacity of a classroom.

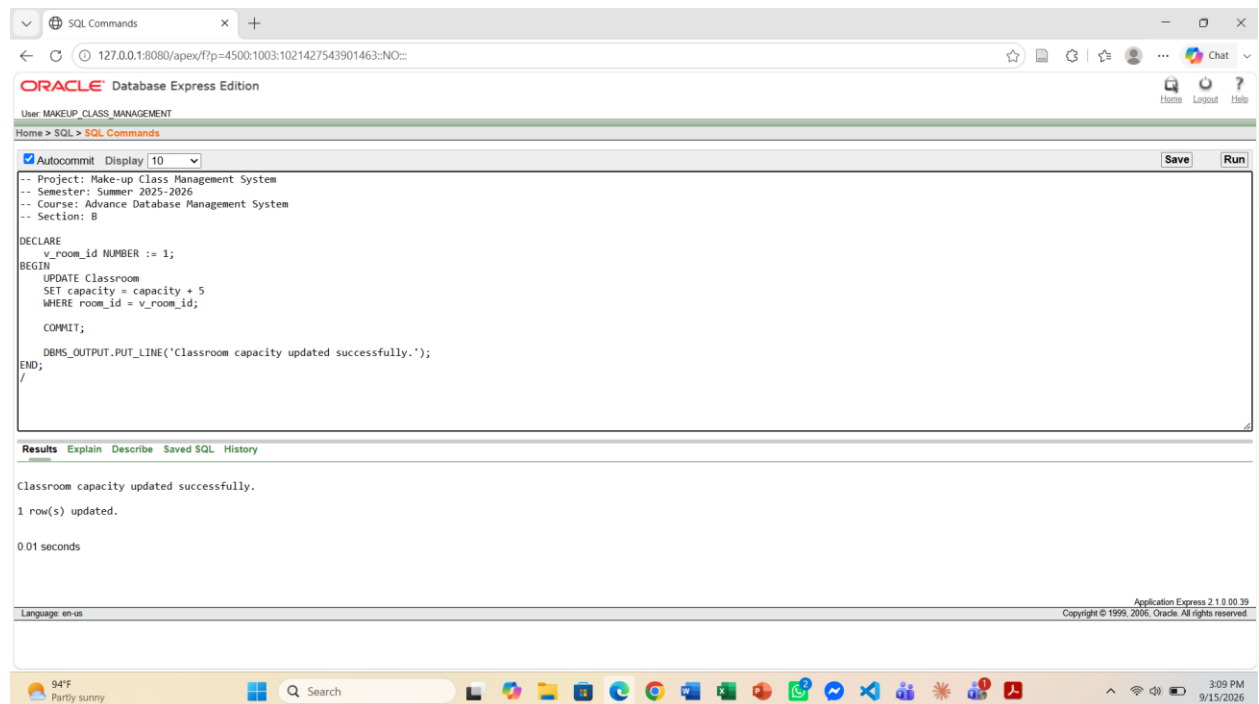
```
DECLARE
    v_room_id NUMBER := 1;
BEGIN
    UPDATE Classroom
    SET capacity = capacity + 5
    WHERE room_id = v_room_id;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Classroom capacity updated successfully.');
```

END;

/



### Explicit Locking:

#### Query 1: Lock a Make-up Class Request

Question:

Write a PL/SQL program using explicit locking to lock a Make-up Class Request before updating its status.

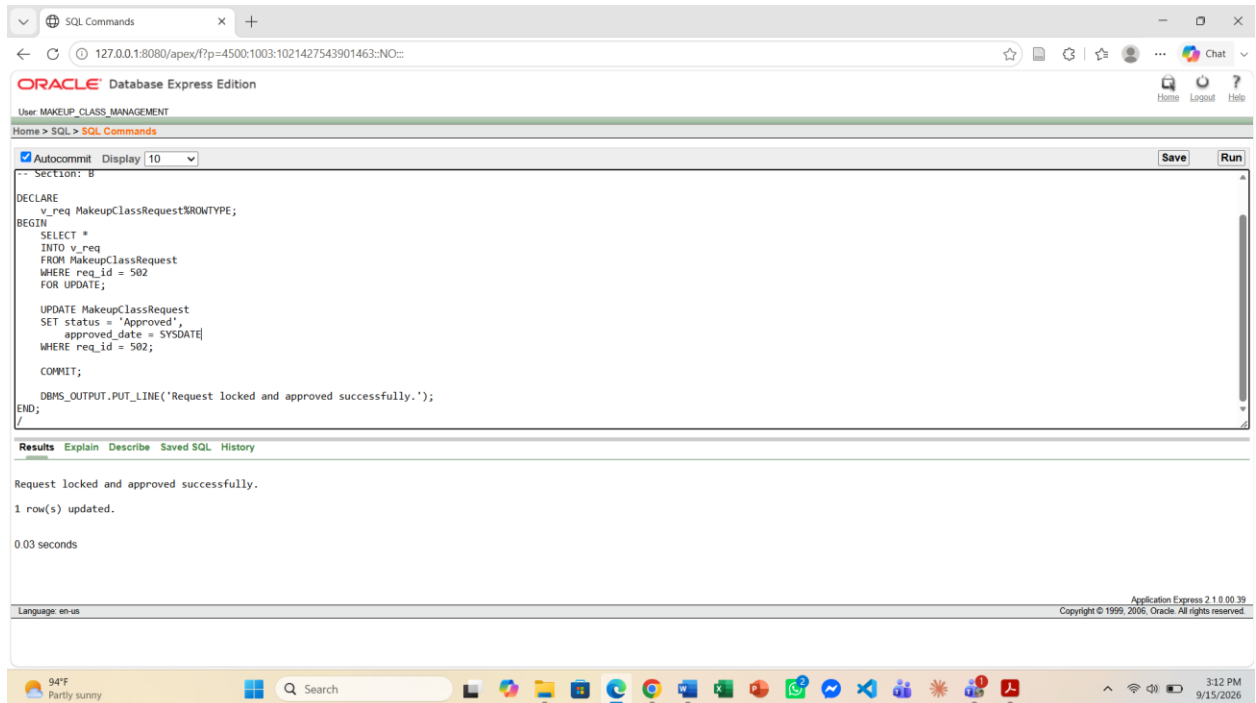
```
DECLARE
    v_req MakeupClassRequest%ROWTYPE;
BEGIN
    SELECT *
    INTO v_req
    FROM MakeupClassRequest
    WHERE req_id = 502
    FOR UPDATE;

    UPDATE MakeupClassRequest
    SET status = 'Approved',
        approved_date = SYSDATE
    WHERE req_id = 502;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Request locked and approved successfully.');
```

END;  
/



## Query 2: Lock a Classroom Before Updating Capacity

Question:

Write a PL/SQL program using explicit locking to lock a classroom before updating its capacity.

DECLARE

    v\_capacity Classroom.capacity%TYPE;

BEGIN

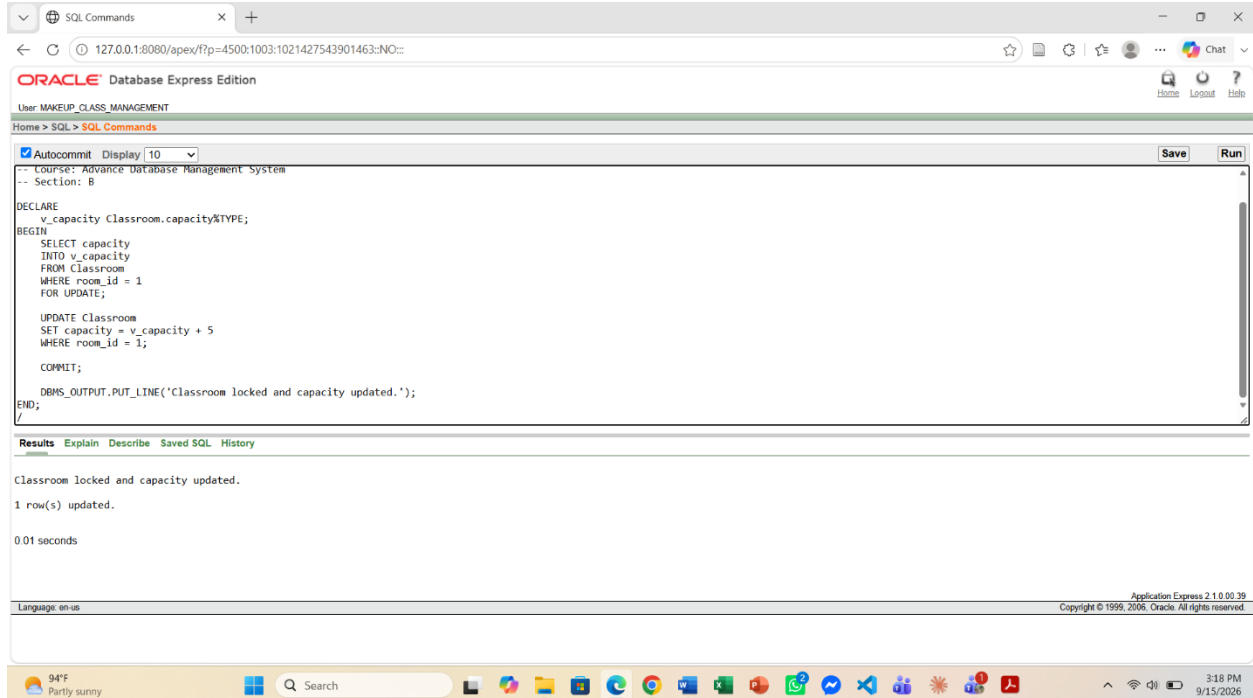
    SELECT capacity  
     INTO v\_capacity  
     FROM Classroom  
     WHERE room\_id = 1  
     FOR UPDATE;

    UPDATE Classroom  
     SET capacity = v\_capacity + 5  
     WHERE room\_id = 1;

COMMIT;

    DBMS\_OUTPUT.PUT\_LINE('Classroom locked and capacity updated.');

END;



## Exception Handling of Advance PL/SQL:

### 1. Stored Function

#### Question 1

Write a PL/SQL stored function to return the name of a student using the Student ID.

```
CREATE OR REPLACE FUNCTION get_student_name(p_sid NUMBER)
```

```
RETURN VARCHAR2
```

```
IS
```

```
  v_name Student.s_name%TYPE;
```

```
BEGIN
```

```
  SELECT s_name
```

```
  INTO v_name
```

```
  FROM Student
```

```
  WHERE s_id = p_sid;
```

```
RETURN v_name;
```

```
EXCEPTION
```

```
WHEN NO_DATA_FOUND THEN
```

```
    DBMS_OUTPUT.PUT_LINE('No student found with ID ' || p_sid);
```

```
    RETURN NULL;
```

```
WHEN TOO_MANY_ROWS THEN
```

```
    DBMS_OUTPUT.PUT_LINE('Multiple students found with ID ' || p_sid);
```

```
    RETURN NULL;
```

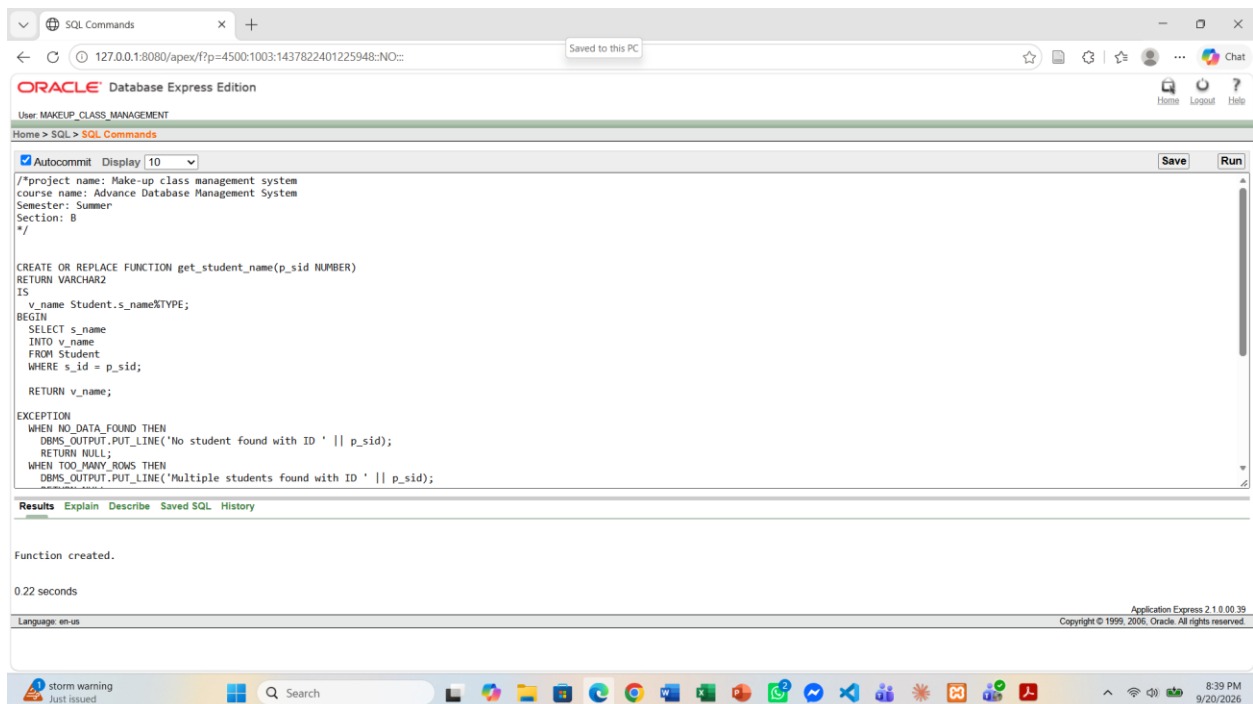
```
WHEN OTHERS THEN
```

```
    DBMS_OUTPUT.PUT_LINE('Error ' || SQLCODE || ': ' || SQLERRM);
```

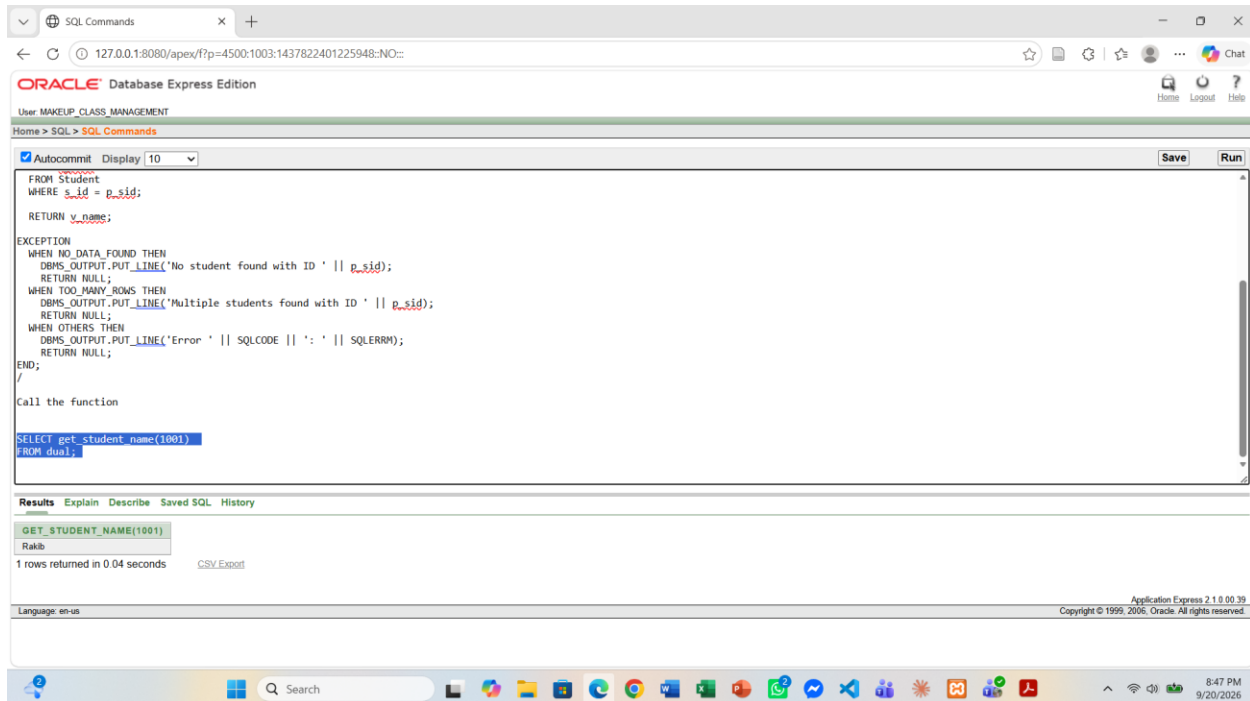
```
    RETURN NULL;
```

```
END;
```

```
/
```



```
SELECT get_student_name(1001) FROM dual;
```



## Question 2

Write a PL/SQL stored function to return the name of a teacher using the Teacher ID.

CREATE OR REPLACE FUNCTION get\_teacher\_name(p\_tid NUMBER)

RETURN VARCHAR2

IS

v\_name Teacher.t\_name%TYPE;

BEGIN

SELECT t\_name

INTO v\_name

FROM Teacher

WHERE t\_id = p\_tid;

RETURN v\_name;

EXCEPTION

WHEN NO\_DATA\_FOUND THEN

DBMS\_OUTPUT.PUT\_LINE('No teacher found with ID ' || p\_tid);

RETURN NULL;

WHEN TOO\_MANY\_ROWS THEN

DBMS\_OUTPUT.PUT\_LINE('Multiple teachers found with ID ' || p\_tid);

RETURN NULL;

WHEN OTHERS THEN

DBMS\_OUTPUT.PUT\_LINE('Error ' || SQLCODE || ':' || SQLERRM);

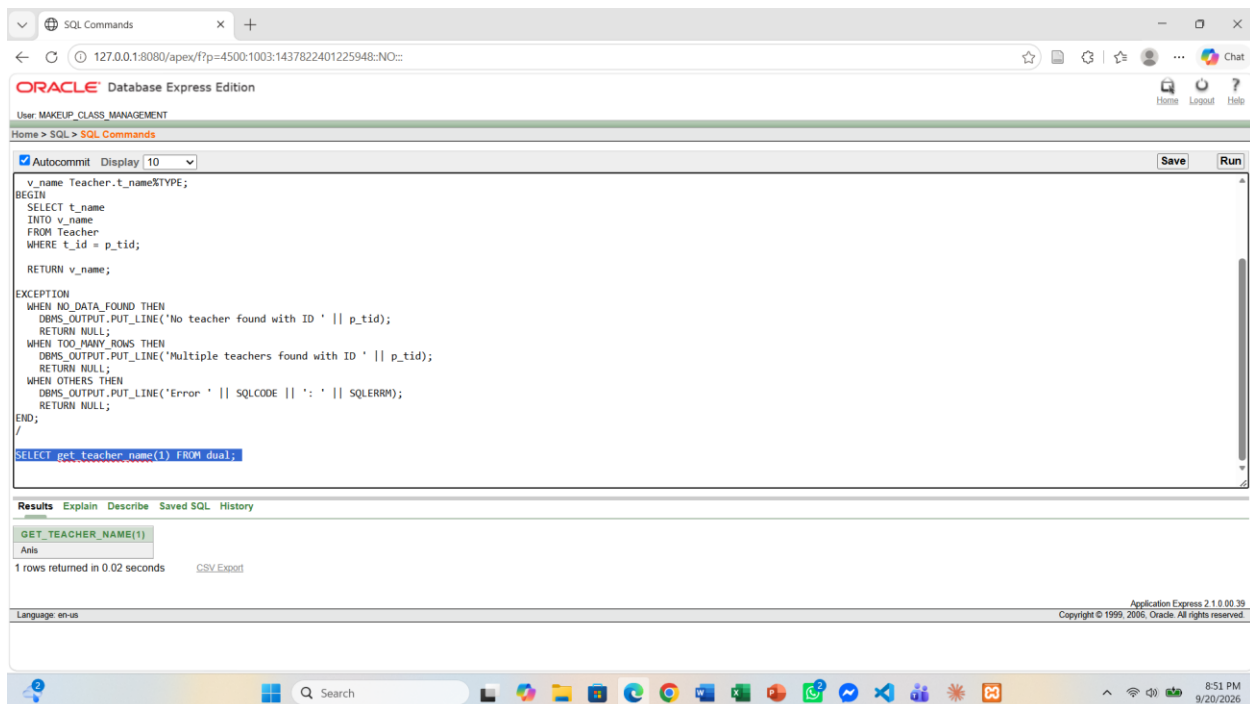
RETURN NULL;

END;

/

## Call the function

SELECT get\_teacher\_name(1) FROM dual;



## 2. Stored Procedure

### Question 1

Write a PL/SQL stored procedure to display the name of a student using Student ID.

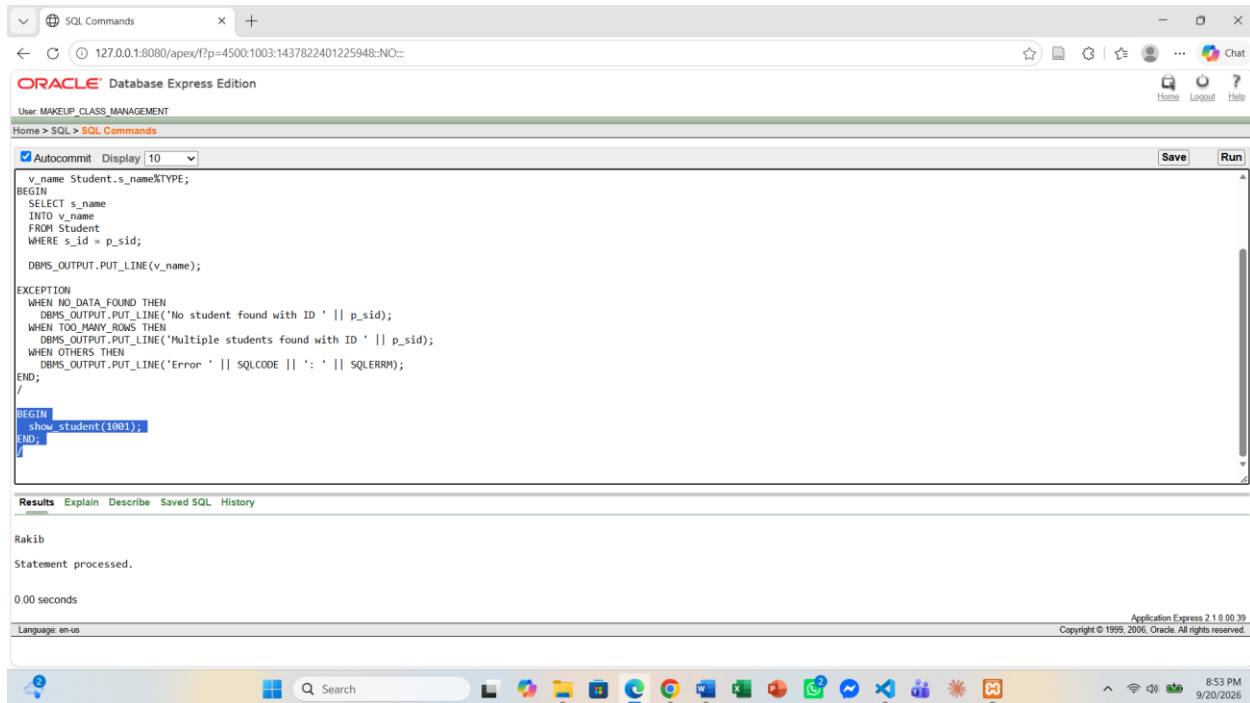
```
CREATE OR REPLACE PROCEDURE show_student(p_sid NUMBER)
IS
    v_name Student.s_name%TYPE;
BEGIN
    SELECT s_name
    INTO v_name
    FROM Student
    WHERE s_id = p_sid;

    DBMS_OUTPUT.PUT_LINE(v_name);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No student found with ID ' || p_sid);
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('Multiple students found with ID ' || p_sid);
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error ' || SQLCODE || ':' || SQLERRM);
END;
/
```

Execute:

```
BEGIN
    show_student(1001);
END;
/
```



## Question 2

Write a PL/SQL stored procedure to update the status of a Make-up Class Request to 'Approved' using Request ID.

CREATE OR REPLACE PROCEDURE update\_status(p\_req NUMBER)

IS

e\_invalid\_request EXCEPTION;

BEGIN

UPDATE MakeupClassRequest

SET status = 'Approved'

WHERE req\_id = p\_req;

IF SQL%NOTFOUND THEN

RAISE e\_invalid\_request;

END IF;

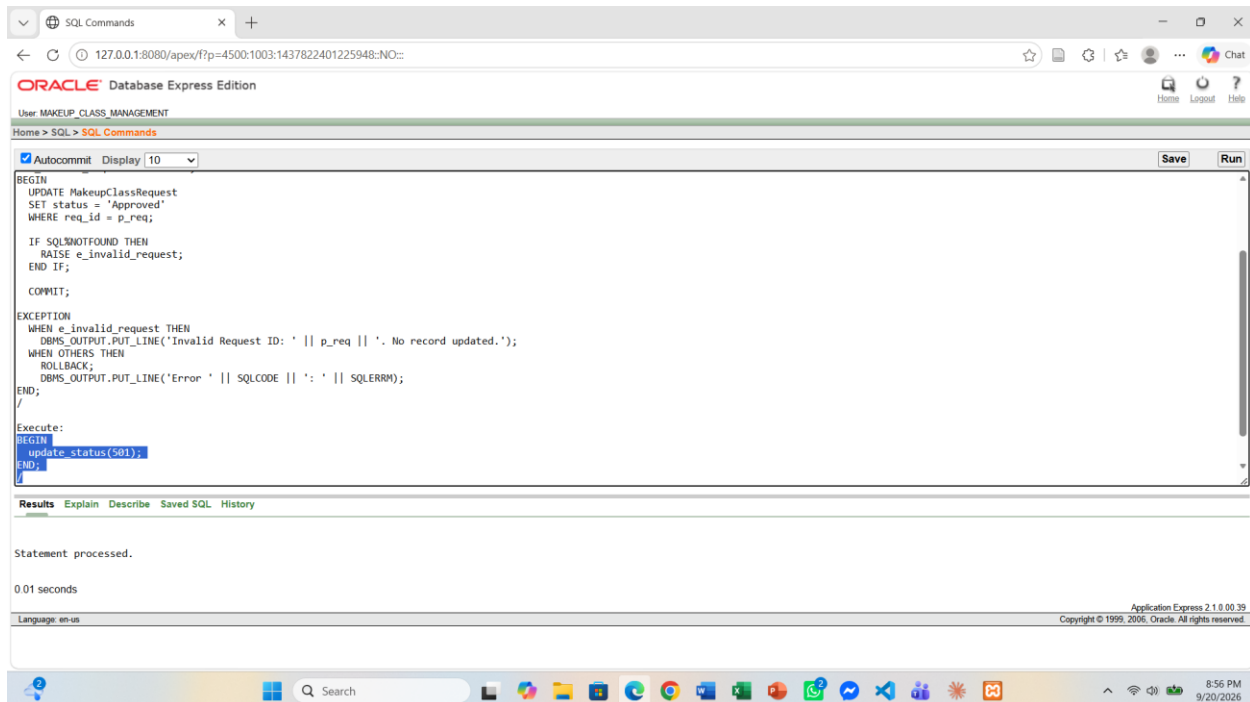
COMMIT;

## EXCEPTION

```
WHEN e_invalid_request THEN  
    DBMS_OUTPUT.PUT_LINE('Invalid Request ID: ' || p_req || '. No record updated.');
```

## WHEN OTHERS THEN

```
    ROLLBACK;  
    DBMS_OUTPUT.PUT_LINE('Error ' || SQLCODE || ': ' || SQLERRM);  
END;  
/  
Execute:  
BEGIN  
    update_status(501);  
END; /
```



## 3. Table-Based Record

### Question 1

Write a PL/SQL program using a table-based record (%ROWTYPE) to retrieve and display a teacher's information.

```

DECLARE

    v_teacher Teacher%ROWTYPE;

BEGIN

    SELECT *

    INTO v_teacher

    FROM Teacher

    WHERE t_id = 1;


    DBMS_OUTPUT.PUT_LINE(v_teacher.t_name);


EXCEPTION

    WHEN NO_DATA_FOUND THEN

        DBMS_OUTPUT.PUT_LINE('No teacher found with the given ID.');
```

```

    WHEN TOO_MANY_ROWS THEN

        DBMS_OUTPUT.PUT_LINE('More than one teacher matched the given ID.');
```

```

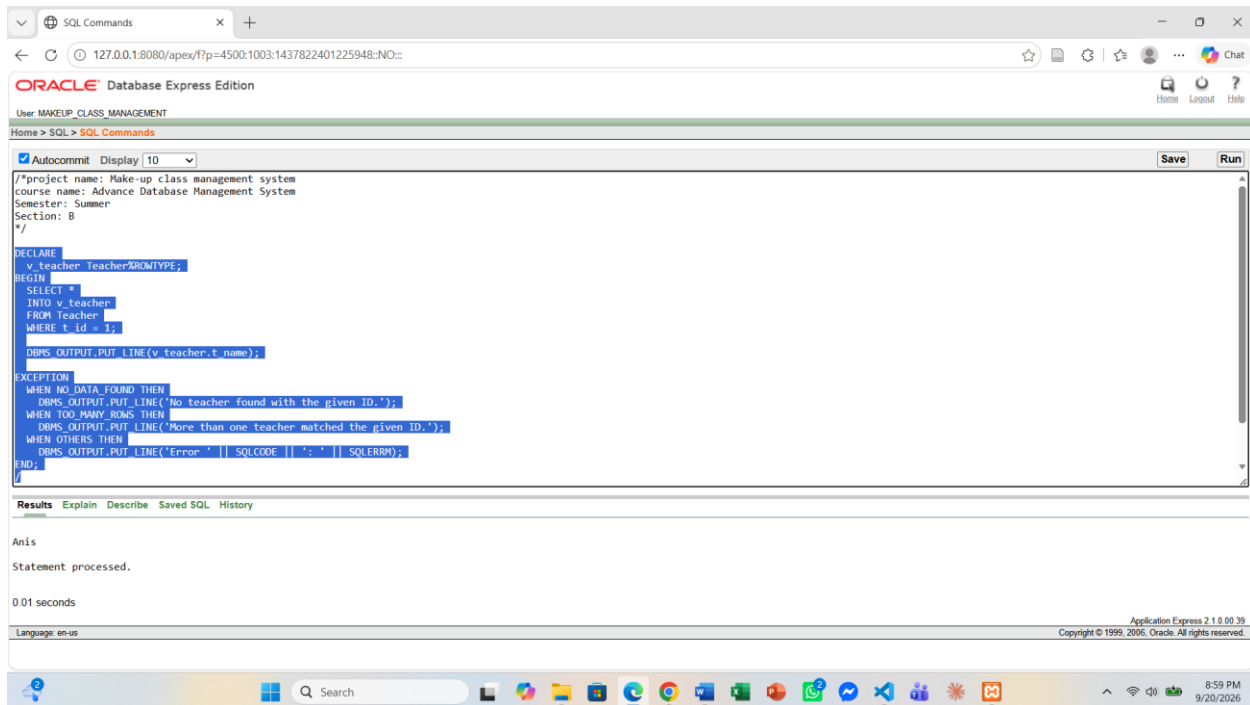
    WHEN OTHERS THEN

        DBMS_OUTPUT.PUT_LINE('Error ' || SQLCODE || ': ' || SQLERRM);

END;

/

```



## Question 2

Write a PL/SQL program using a table-based record (%ROWTYPE) to retrieve and display a student's information.

DECLARE

    v\_student Student%ROWTYPE;

BEGIN

    SELECT \*

    INTO v\_student

    FROM Student

    WHERE s\_id = 1001;

    DBMS\_OUTPUT.PUT\_LINE(v\_student.s\_name);

EXCEPTION

    WHEN NO\_DATA\_FOUND THEN

        DBMS\_OUTPUT.PUT\_LINE('No student found with the given ID.');

    WHEN TOO\_MANY\_ROWS THEN

```

        DBMS_OUTPUT.PUT_LINE('More than one student matched the given ID.');
```

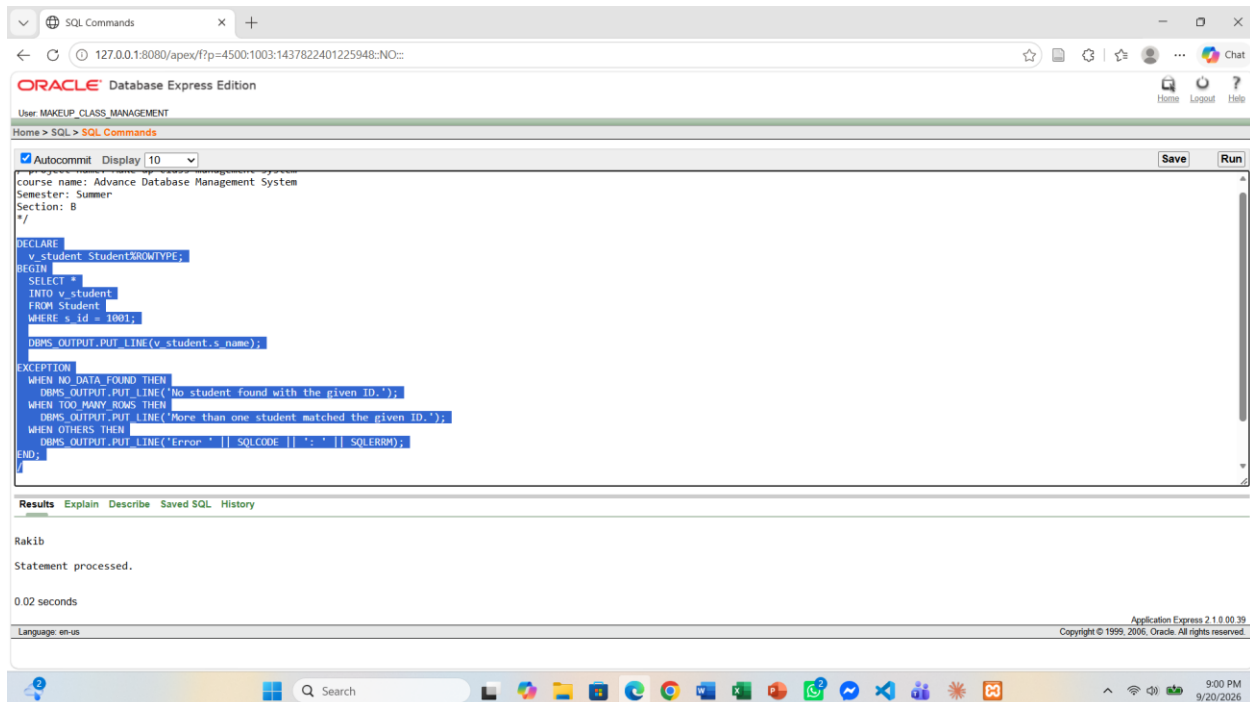
WHEN OTHERS THEN

```

        DBMS_OUTPUT.PUT_LINE('Error ' || SQLCODE || ':' || SQLERRM);
```

END;

/



#### 4. Explicit Cursor

##### Question 1

Write a PL/SQL program using an explicit cursor to display the names of all students.

```

DECLARE

  CURSOR c IS SELECT s_name FROM Student;

  v_name Student.s_name%TYPE;

BEGIN

  OPEN c;

  LOOP

    FETCH c INTO v_name;

    EXIT WHEN c%NOTFOUND;
```

```

        DBMS_OUTPUT.PUT_LINE(v_name);

    END LOOP;

    CLOSE c;

EXCEPTION

    WHEN OTHERS THEN

        IF c%ISOPEN THEN

            CLOSE c;

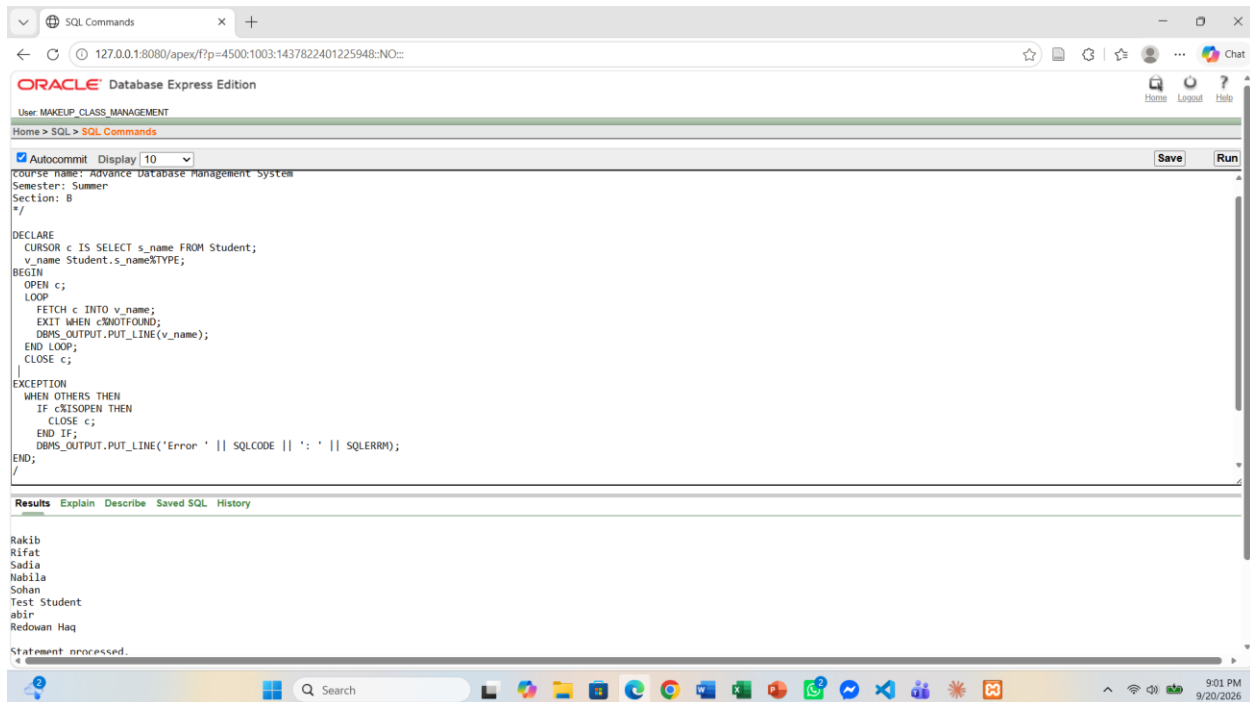
        END IF;

        DBMS_OUTPUT.PUT_LINE('Error ' || SQLCODE || ':' || SQLERRM);

END;

/

```



## Question 2

Write a PL/SQL program using an explicit cursor to display the names of all teachers.

```

DECLARE

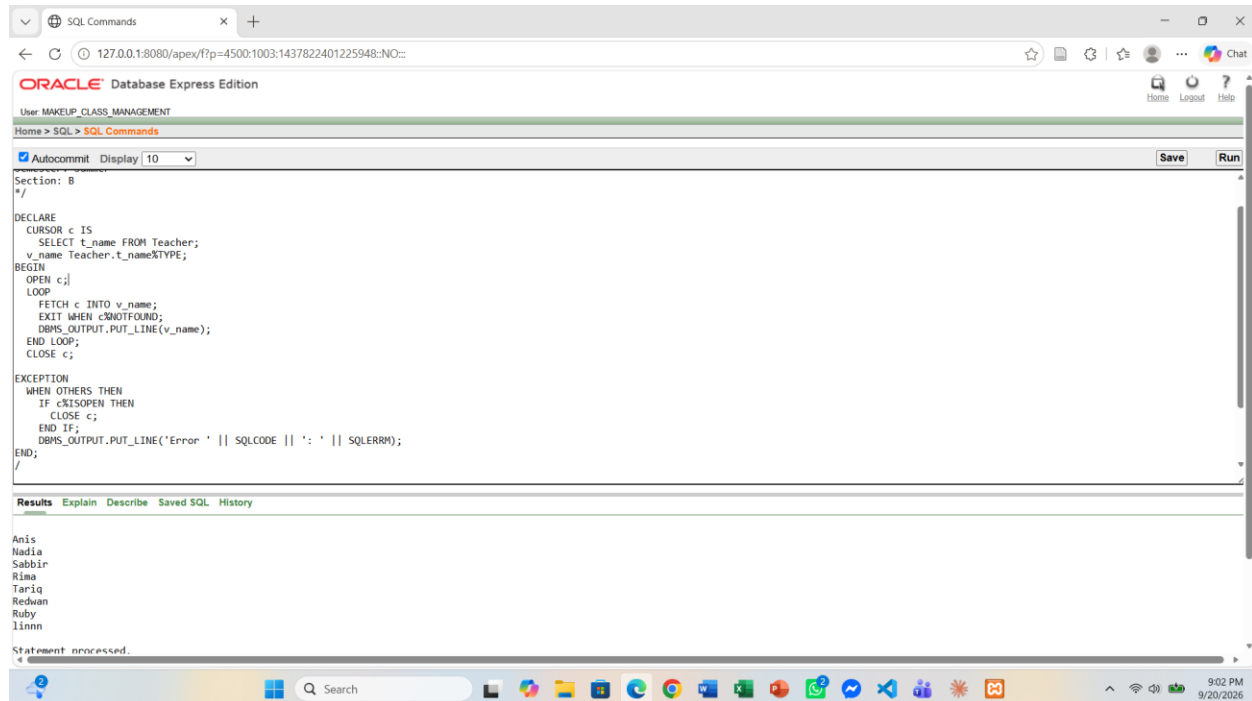
```

```

CURSOR c IS
    SELECT t_name FROM Teacher;
    v_name Teacher.t_name%TYPE;
BEGIN
    OPEN c;
    LOOP
        FETCH c INTO v_name;
        EXIT WHEN c%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(v_name);
    END LOOP;
    CLOSE c;

EXCEPTION
    WHEN OTHERS THEN
        IF c%ISOPEN THEN
            CLOSE c;
        END IF;
        DBMS_OUTPUT.PUT_LINE('Error ' || SQLCODE || ':' || SQLERRM);
END;
/

```



## 5. Cursor-Based Record

### Question 1

Write a PL/SQL program using a cursor-based record to display all student names.

DECLARE

CURSOR c IS

SELECT \* FROM Student;

rec c%ROWTYPE;

BEGIN

OPEN c;

LOOP

FETCH c INTO rec;

EXIT WHEN c%NOTFOUND;

DBMS\_OUTPUT.PUT\_LINE(rec.s\_name);

END LOOP;

CLOSE c;

EXCEPTION

WHEN OTHERS THEN

IF c%ISOPEN THEN

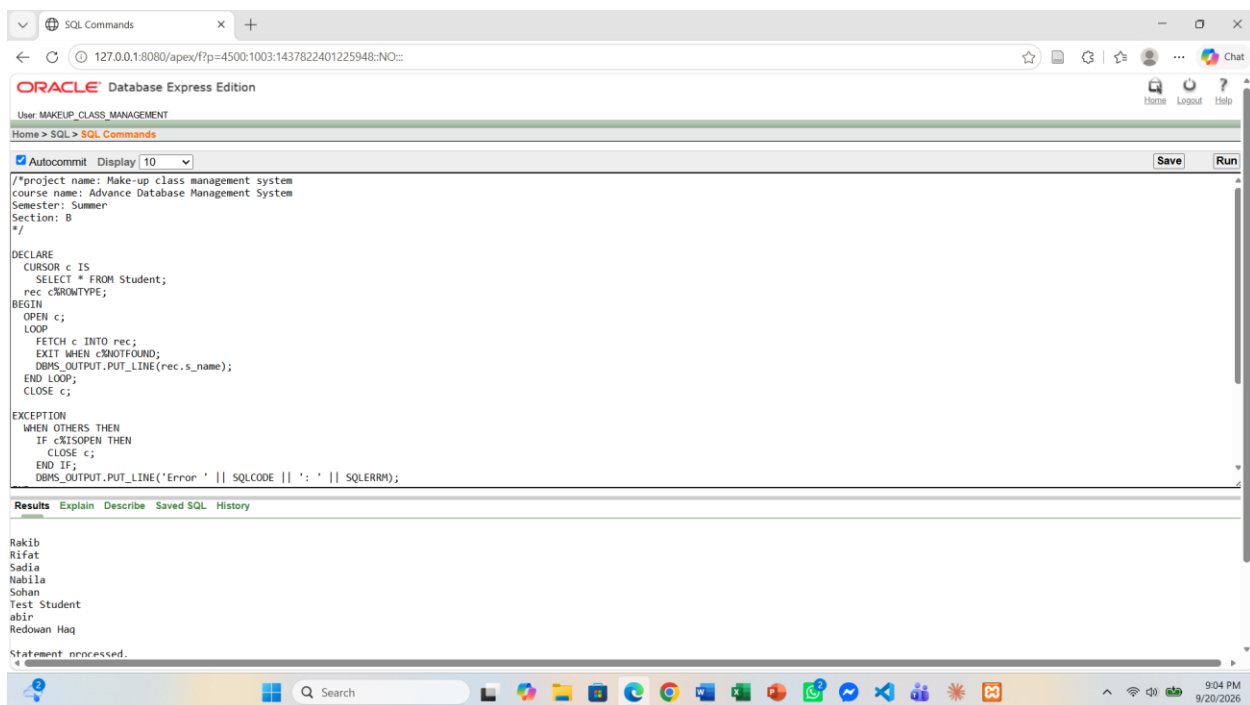
CLOSE c;

END IF;

DBMS\_OUTPUT.PUT\_LINE('Error ' || SQLCODE || ': ' || SQLERRM);

END;

/



## Question 2

Write a PL/SQL program using a cursor-based record to display all teacher names.

DECLARE

CURSOR c IS

SELECT \* FROM Teacher;

rec c%ROWTYPE;

BEGIN

```
OPEN c;
```

```
LOOP
```

```
    FETCH c INTO rec;
```

```
    EXIT WHEN c%NOTFOUND;
```

```
    DBMS_OUTPUT.PUT_LINE(rec.t_name);
```

```
END LOOP;
```

```
CLOSE c;
```

```
EXCEPTION
```

```
    WHEN OTHERS THEN
```

```
        IF c%ISOPEN THEN
```

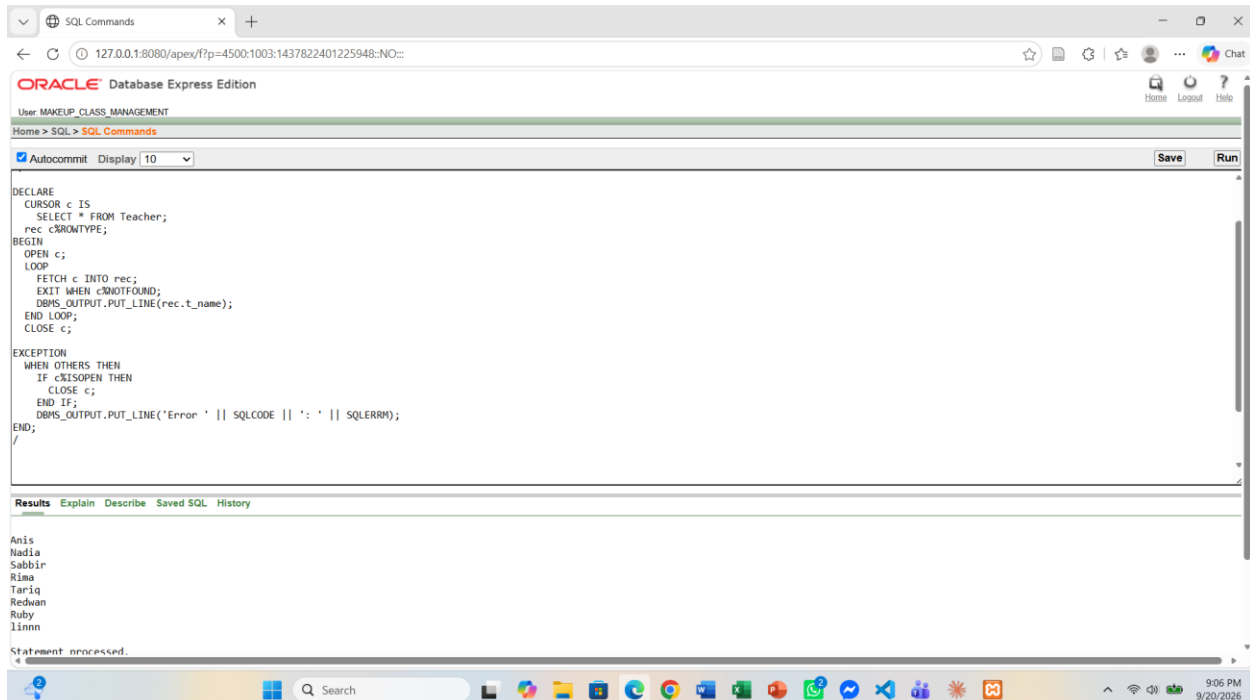
```
            CLOSE c;
```

```
        END IF;
```

```
        DBMS_OUTPUT.PUT_LINE('Error ' || SQLCODE || ' : ' || SQLERRM);
```

```
END;
```

```
/
```



## 6. Row-Level Trigger

### Question 1

Write a row-level trigger that displays a message whenever a new student is inserted into the Student table.

```
CREATE OR REPLACE TRIGGER trg_student_insert
```

```
AFTER INSERT ON Student
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    DBMS_OUTPUT.PUT_LINE('Student Added');
```

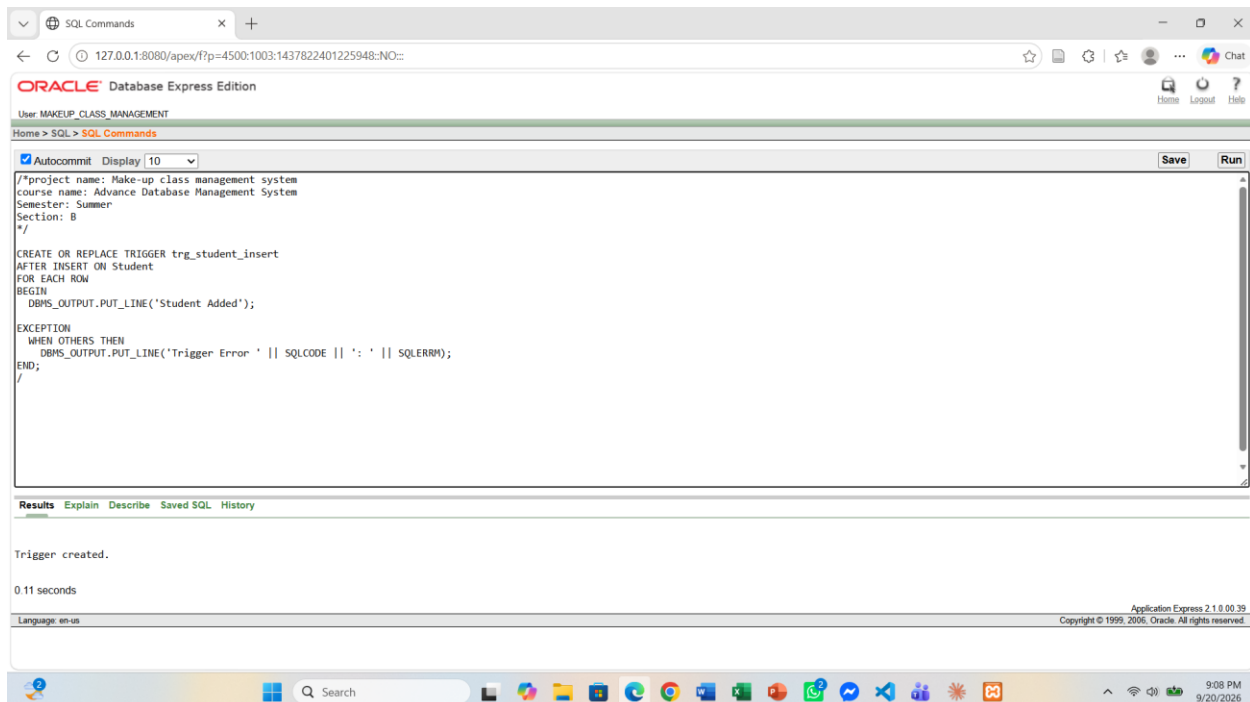
```
EXCEPTION
```

```
    WHEN OTHERS THEN
```

```
        DBMS_OUTPUT.PUT_LINE('Trigger Error ' || SQLCODE || ': ' || SQLERRM);
```

```
END;
```

```
/
```

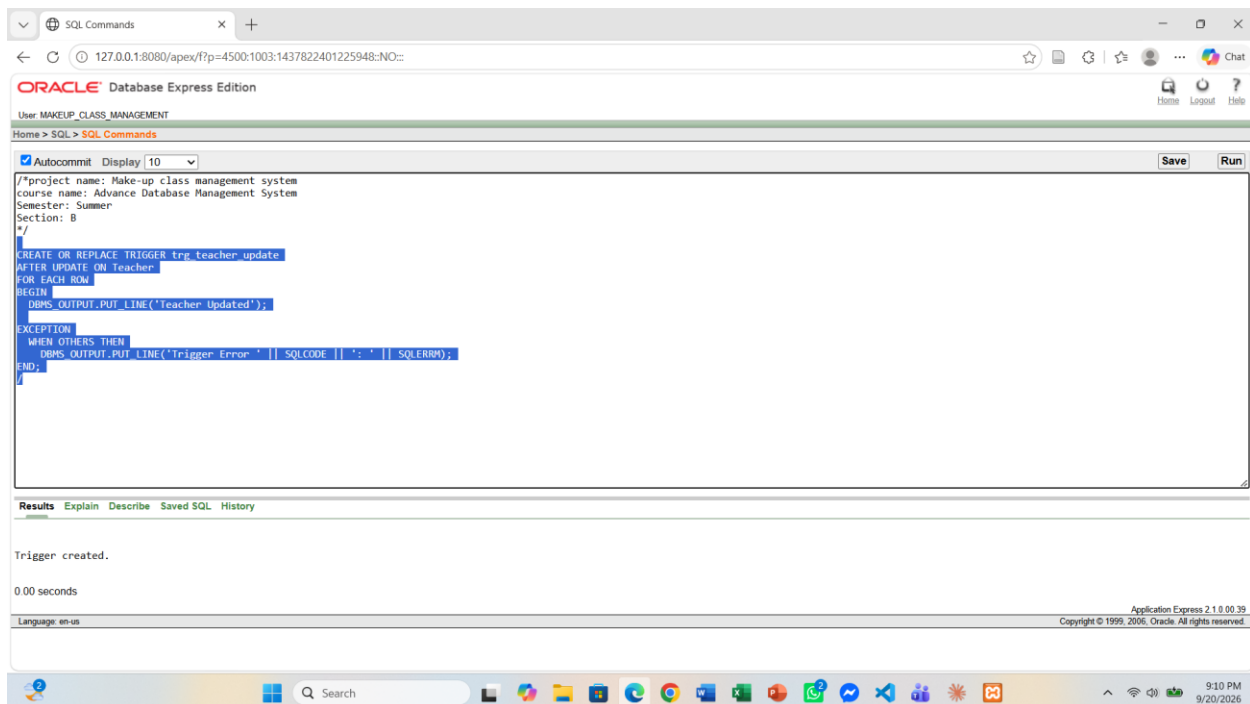


### Question 2

Write a row-level trigger that displays a message whenever a teacher record is updated.

```
CREATE OR REPLACE TRIGGER trg_teacher_update
AFTER UPDATE ON Teacher
FOR EACH ROW
BEGIN
    DBMS_OUTPUT.PUT_LINE('Teacher Updated');

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Trigger Error ' || SQLCODE || ': ' || SQLERRM);
END;
/
```



## 7. Statement-Level Trigger

### Question 1

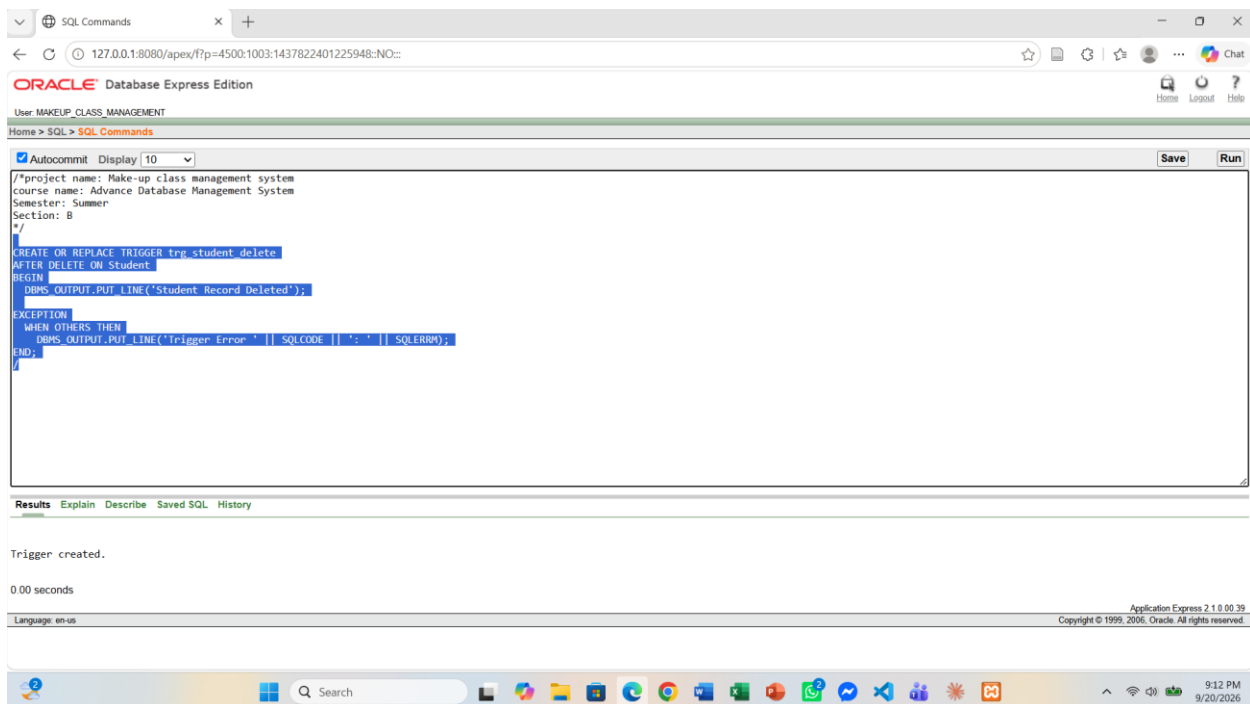
Write a statement-level trigger that displays a message whenever a record is deleted from the Student table.

```

CREATE OR REPLACE TRIGGER trg_student_delete
AFTER DELETE ON Student
BEGIN
    DBMS_OUTPUT.PUT_LINE('Student Record Deleted');

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Trigger Error ' || SQLCODE || ': ' || SQLERRM);
END;
/

```



## Question 2

Write a statement-level trigger that displays a message whenever a new teacher record is inserted into the Teacher table.

```

CREATE OR REPLACE TRIGGER trg_teacher_insert
AFTER INSERT ON Teacher
BEGIN
    DBMS_OUTPUT.PUT_LINE('Teacher Table Modified');

```

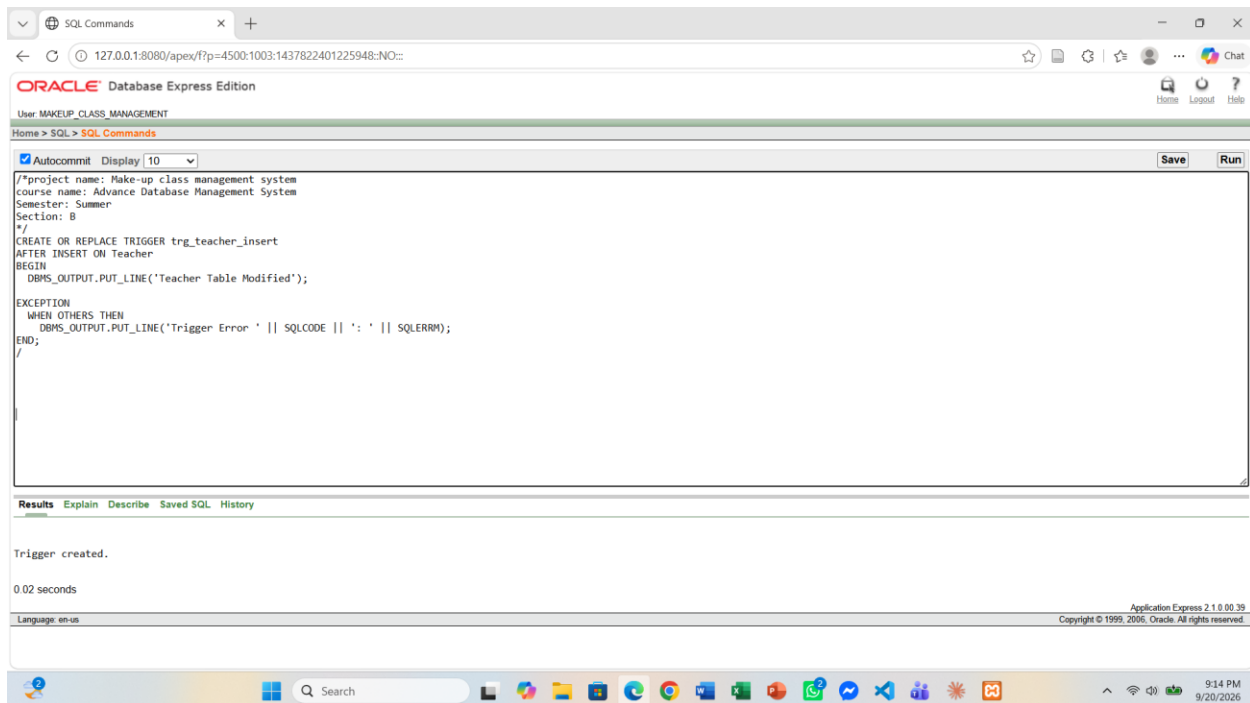
EXCEPTION

WHEN OTHERS THEN

DBMS\_OUTPUT.PUT\_LINE('Trigger Error ' || SQLCODE || ': ' || SQLERRM);

END;

/



## 8. Package

### Question 1

Write a PL/SQL package named `student_pkg` that contains a procedure to display a student's name using Student ID.

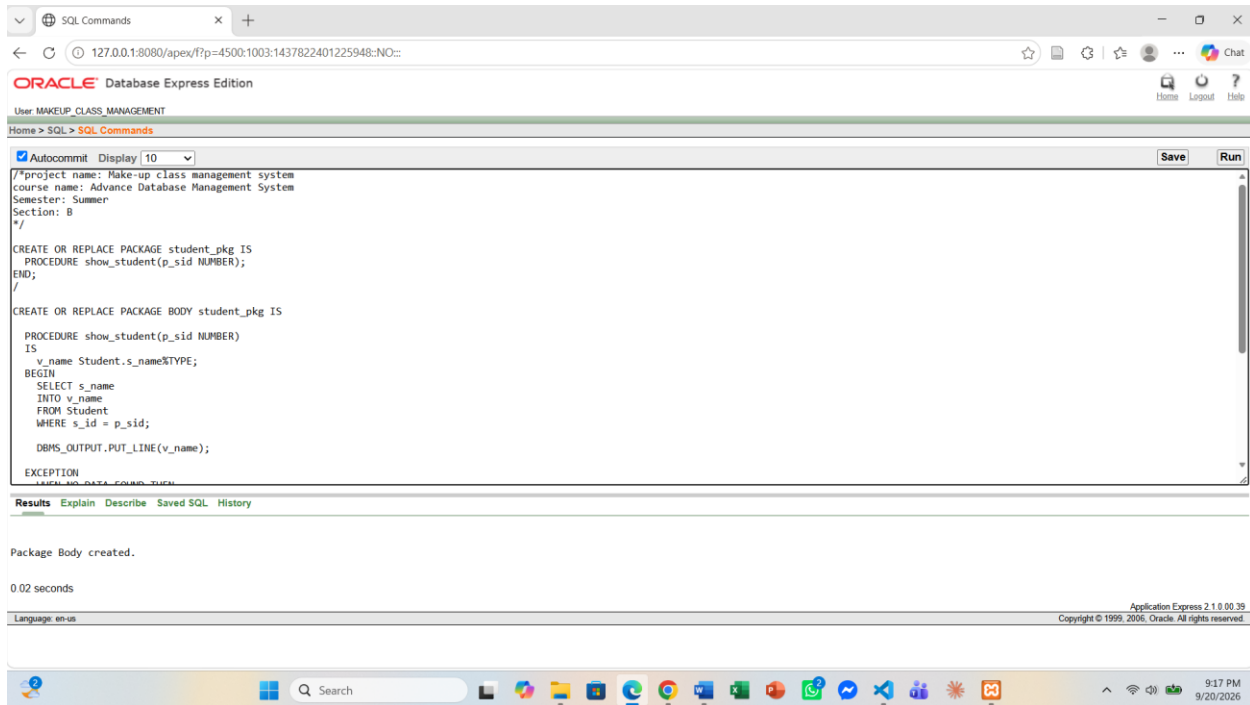
Package specification:

CREATE OR REPLACE PACKAGE `student_pkg` IS

```
PROCEDURE show_student(p_sid NUMBER);  
END;  
/
```

Package body:

```
CREATE OR REPLACE PACKAGE BODY student_pkg IS  
  PROCEDURE show_student(p_sid NUMBER)  
  IS  
    v_name Student.s_name%TYPE;  
  BEGIN  
    SELECT s_name  
    INTO v_name  
    FROM Student  
    WHERE s_id = p_sid;  
    DBMS_OUTPUT.PUT_LINE(v_name);  
  EXCEPTION  
    WHEN NO_DATA_FOUND THEN  
      DBMS_OUTPUT.PUT_LINE('No student found with ID ' || p_sid);  
    WHEN TOO_MANY_ROWS THEN  
      DBMS_OUTPUT.PUT_LINE('Multiple students found with ID ' || p_sid);  
    WHEN OTHERS THEN  
      DBMS_OUTPUT.PUT_LINE('Error ' || SQLCODE || ': ' || SQLERRM);  
  END show_student;  
END student_pkg;  
/
```



## Question 2

Write a PL/SQL package named `teacher_pkg` that contains a function to return the total number of teachers.

Package specification:

```

CREATE OR REPLACE PACKAGE teacher_pkg IS
  FUNCTION total_teacher RETURN NUMBER;
END;
/

```

Package body:

```

CREATE OR REPLACE PACKAGE BODY teacher_pkg IS
  FUNCTION total_teacher
  RETURN NUMBER
  IS
    total NUMBER;
  BEGIN

```

```

SELECT COUNT(*)

INTO total

FROM Teacher;

RETURN total;

EXCEPTION

WHEN NO_DATA_FOUND THEN

    DBMS_OUTPUT.PUT_LINE('No teacher records found.');
```

```

RETURN 0;

WHEN OTHERS THEN

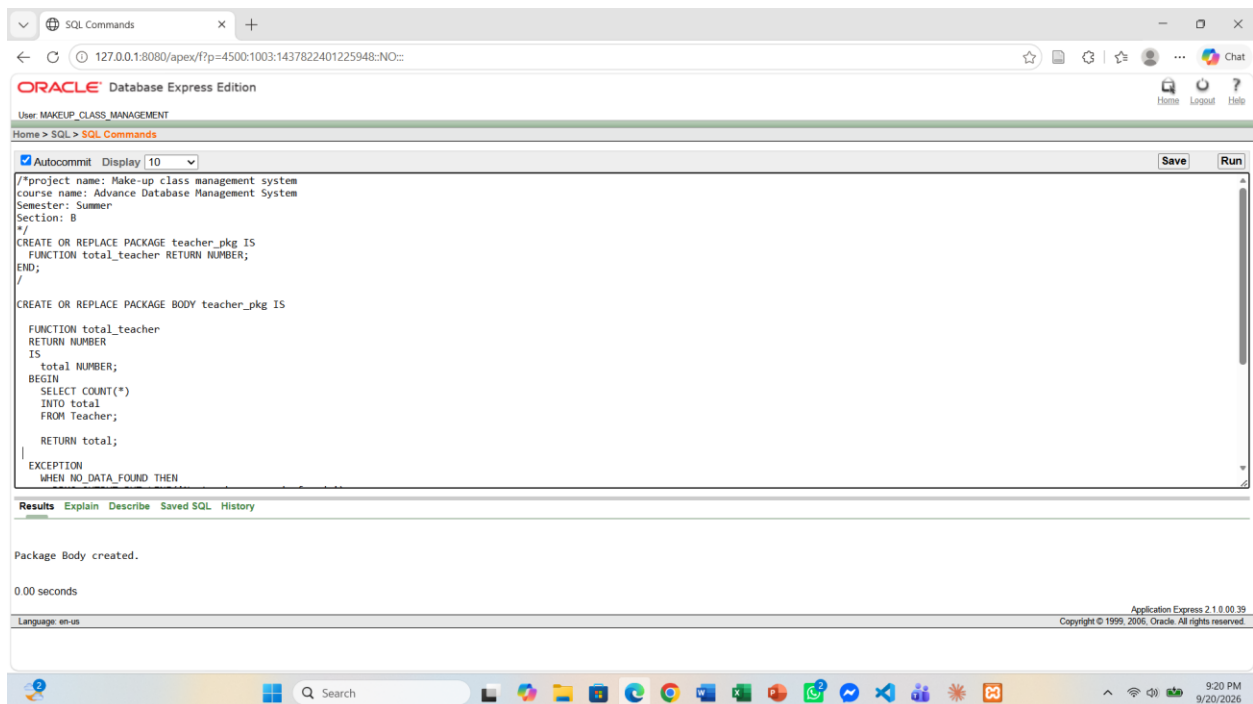
    DBMS_OUTPUT.PUT_LINE('Error ' || SQLCODE || ': ' || SQLERRM);

RETURN NULL;

END total_teacher;

END teacher_pkg;

/
```



### **Relational Algebra:**

#### **Query 1 — Find all students of DBMS class**

Question:

Find the names and IDs of all students who are enrolled in the **DBMS** class.

### **Relational Algebra:**

$$\pi_{s\_id, s\_name}(\sigma_{course\_name='DBMS'}(Student \bowtie_{Student.class\_id=Class.class\_id} Class))$$

#### **Query 2 — Find teacher of the DBMS class**

Question:

Find the name of the teacher who teaches the **DBMS** class.

### **Relational Algebra:**

$$\pi_{t\_name}(\sigma_{course\_name='DBMS'}(Class \bowtie_{Class.t\_id=Teacher.t\_id} Teacher))$$

#### **Query 3 — Find all approved make-up class requests**

Question:

Find the request ID, requested date, and proposed time of all approved make-up class requests.

### **Relational Algebra:**

$$\pi_{req\_id, requested\_date, proposed\_time}(\sigma_{status='Approved'}(MakeupClassRequest))$$

#### **Query 4 — Find make-up requests with classroom information**

Question:

Find the request ID, building name, and classroom capacity for each make-up class

Request.

### **Relational Algebra:**

$$\pi_{req\_id, building\_name, capacity}(MakeupClassRequest \bowtie_{MakeupClassRequest.room\_id=Classroom.room\_id} Classroom)$$

#### **Query 5 — Find students who received notifications.**

Question:

Find the names of students who received notifications about make-up classes.

### **Relational Algebra:**

$$\pi_{s\_id, s\_name}(Notification \bowtie_{Notification.s\_id=Student.s\_id} Student)$$